

Chapter 6: Pattern Mining

Basic Concepts and Methods

- ☐ Basic Concepts 
- ☐ Frequent Itemset Mining Methods
- ☐ Which Patterns Are Interesting?—Pattern Evaluation Methods
- ☐ Summary

What Are Patterns?

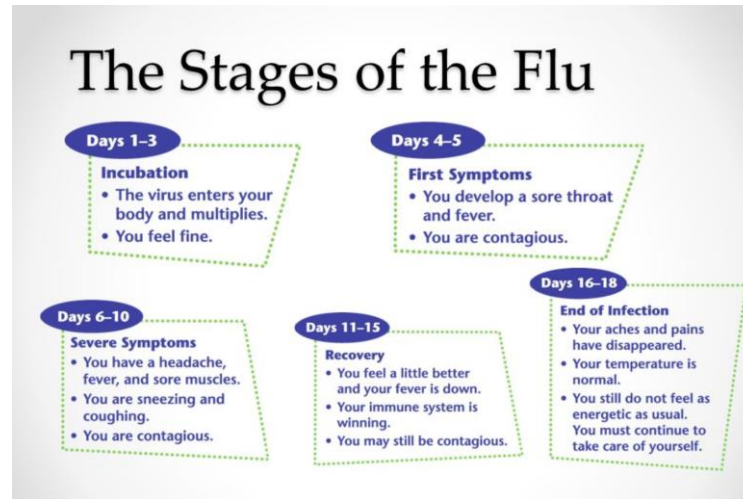
❑ What are patterns?

- ❑ **Patterns:** A set of items, subsequences, or substructures that occur frequently together (or strongly correlated) in a data set
- ❑ Patterns represent **intrinsic** and **important properties** of datasets
bản chất

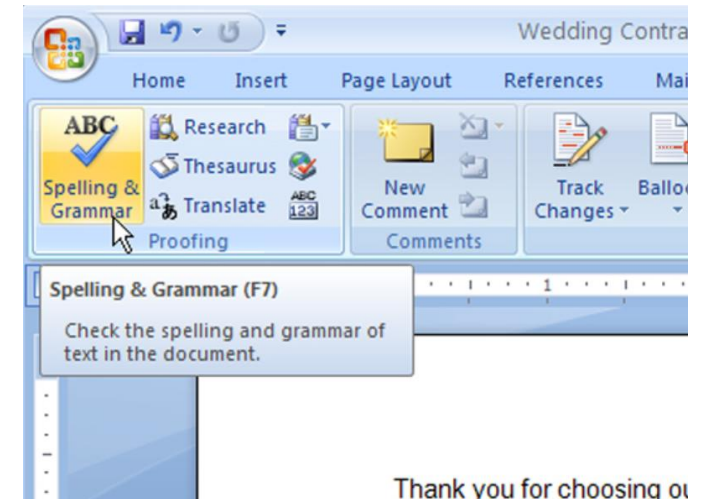


Frequent item set

frequent=xuất hiện thường xuyên (cùng nh



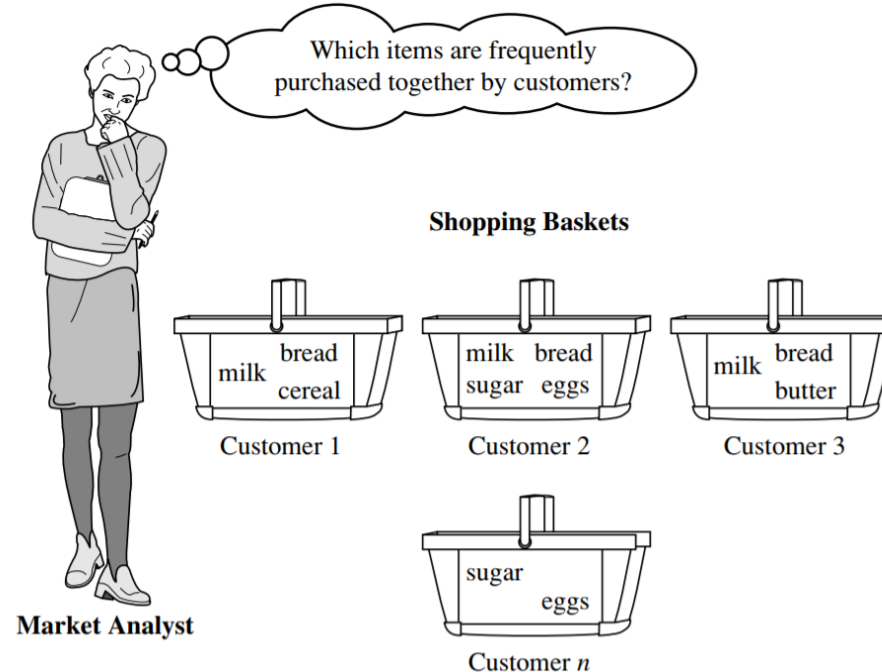
Frequent sequences



Frequent structures

What Is Pattern Discovery?

- ❑ **Pattern discovery**: Uncovering patterns from massive data sets
- ❑ It can answer questions such as:
 - ❑ What products were often purchased together?
 - ❑ What are the subsequent purchases after buying an iPad?



Pattern Discovery: Why Is It Important?

- ❑ **Foundation** for many essential data mining tasks
 - ❑ Association, correlation, and causality analysis
 - ❑ Mining **sequential**, structural (e.g., sub-graph) patterns
 - ❑ **Classification**: Discriminative pattern-based analysis
 - ❑ **Cluster** analysis: Pattern-based subspace clustering
- ❑ Broad **applications**
 - ❑ Market basket analysis, cross-marketing, catalog design, sale campaign analysis, Web log analysis, biological sequence analysis
 - ❑ Many types of data: spatiotemporal, multimedia, time-series, and stream data

Basic Concepts: Transactional Database

□ Transactional Database (TDB)

- Each transaction is associated with an identifier, called a TID.
- May also have counts associated with each item sold

Tid	Items bought
1	Beer, Nuts, Diaper
2	Beer, Coffee, Diaper
3	Beer, Diaper, Eggs
4	Nuts, Eggs, Milk
5	Nuts, Coffee, Diaper, Eggs, Milk

Basic Concepts: k-Itemsets and Their Supports

- Itemset: A set of one or more items

$$I = \{I_1, I_2, \dots, I_m\}$$

tập tất cả các mặt hàng

- k-itemset: An itemset containing k items:

$$X = \{x_1, \dots, x_k\}$$

tập chứa k mặt hàng

- Ex. {Beer, Nuts, Diaper} is a 3-itemset

độ đo cứng

- Absolute support (count) số lần xuất hiện của mặt hàng

- $\text{sup}\{X\}$ = occurrences of an itemset X

- Ex. $\text{sup}\{\text{Beer}\} = 3$

- Ex. $\text{sup}\{\text{Diaper}\} = 4$

- Ex. $\text{sup}\{\text{Beer, Diaper}\} = 3$

- Ex. $\text{sup}\{\text{Beer, Eggs}\} = 1$

Tid	Items bought
1	Beer, Nuts, Diaper
2	Beer, Coffee, Diaper
3	Beer, Diaper, Eggs
4	Nuts, Eggs, Milk
5	Nuts, Coffee, Diaper, Eggs, Milk

- Relative support

- $s\{X\}$ = The fraction of transactions that contains X (i.e., the probability that a transaction contains X)

- Ex. $s\{\text{Beer}\} = 3/5 = 60\%$

- Ex. $s\{\text{Diaper}\} = 4/5 = 80\%$

- Ex. $s\{\text{Beer, Eggs}\} = 1/5 = 20\%$

Basic Concepts: Frequent Itemsets (Patterns)

- An itemset (or a pattern) X is *frequent* if the support of X is no less than a *minsup* threshold σ ngưỡng ngta sẽ cung cấp

- Let $\sigma = 50\%$ (σ : *minsup* threshold) for the given 5-transaction dataset



Tid	Items bought
1	Beer, Nuts, Diaper
2	Beer, Coffee, Diaper
3	Beer, Diaper, Eggs
4	Nuts, Eggs, Milk
5	Nuts, Coffee, Diaper, Eggs, Milk

- All the frequent 1-itemsets:

- Beer: 3/5 (60%); Nuts: 3/5 (60%);
Diaper: 4/5 (80%); Eggs: 3/5 (60%)

- All the frequent 2-itemsets:

- {Beer, Diaper}: 3/5 (60%)

- All the frequent 3-itemsets?

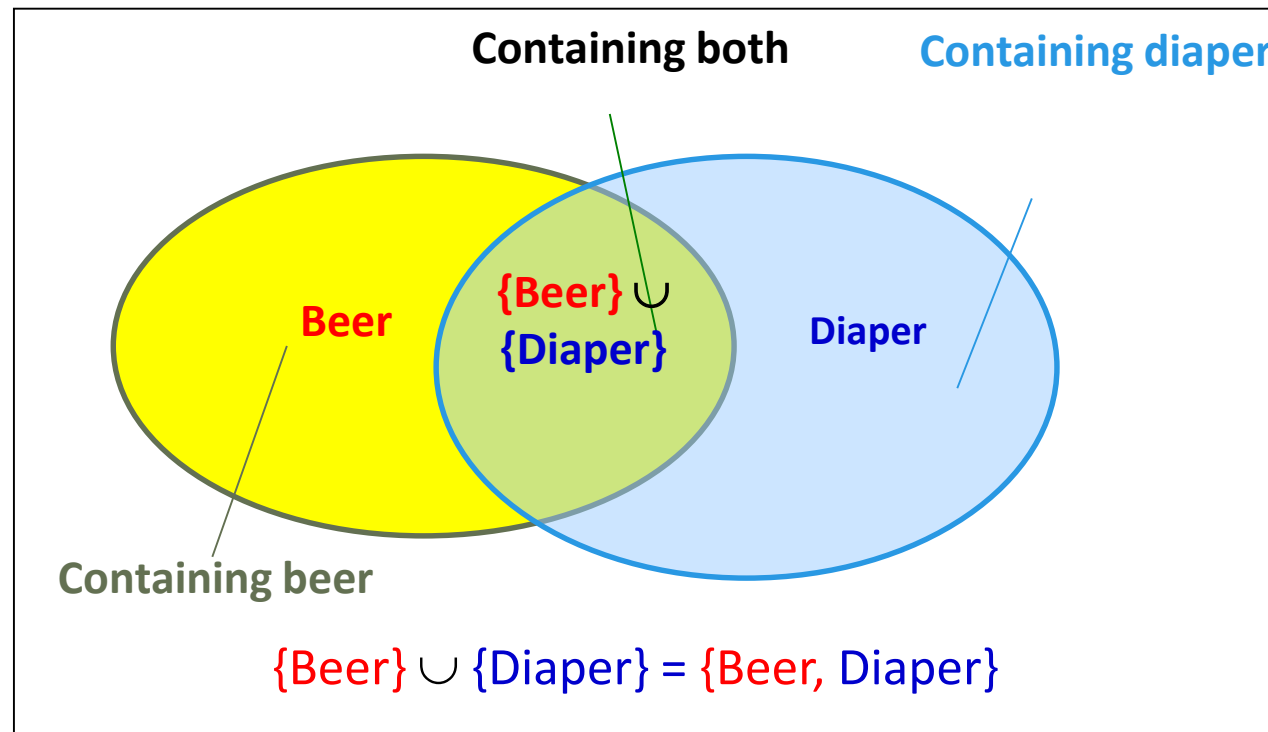
- None

- Why do these itemsets (shown on the left) form the complete set of frequent k -itemsets (patterns) for any k ?

- **Observation:** We may need an efficient method to mine a complete set of frequent patterns

From Frequent Itemsets to Association Rules

- Compared with itemsets, association rules can be more telling
 - Ex. *Diaper* $\rightarrow$ *Beer*
 - *Buying diapers may likely lead to buying beers*



Note: $X \cup Y$: the union of two itemsets

■ The set contains both X and Y

Association Rules

- How do we compute the strength of an association rule $X \rightarrow Y$ (Both X and Y are itemsets)?
- We first compute the following two metrics, s and c .

- **Support of $X \cup Y$**

- Ex. $s\{\text{Diaper, Beer}\} = 3/5 = 0.6$ (i.e., 60%)

- **Confidence of $X \rightarrow Y$** (input đi kèm với minsup)

- The **conditional probability** that a transaction containing X also contains Y :

$$c = \text{sup}(X, Y) / \text{sup}(X) \quad \text{mua X có khả năng mua Y với xác suất (Confident) là bao nhi}$$

- Ex. $c = \text{sup}\{\text{Diaper, Beer}\} / \text{sup}\{\text{Diaper}\} = 3/4 = 0.75$

- In pattern analysis, we are often interested in those rules that dominate the database, and these two metrics ensure the popularity and correlation of X and Y .

Tid	Items bought
1	Beer, Nuts, Diaper
2	Beer, Coffee, Diaper
3	Beer, Diaper, Eggs
4	Nuts, Eggs, Milk
5	Nuts, Coffee, Diaper, Eggs, Milk

Mining Frequent Itemsets and Association Rules

□ Association rule mining

- Given two thresholds: $minsup$, $minconf$
- Find **all** of the rules, $X \rightarrow Y (s, c)$ such that $s \geq minsup$ and $c \geq minconf$

□ Let $minsup = 50\%$

- Freq. 1-itemsets: Beer: 3, Nuts: 3, Diaper: 4, Eggs: 3
- Freq. 2-itemsets: {Beer, Diaper}: 3

□ Let $minconf = 50\%$

- $Beer \rightarrow Diaper$ (60%, 100%)
- $Diaper \rightarrow Beer$ (60%, 75%)



Tid	Items bought
1	Beer, Nuts, Diaper
2	Beer, Coffee, Diaper
3	Beer, Diaper, Eggs
4	Nuts, Eggs, Milk
5	Nuts, Coffee, Diaper, Eggs, Milk

□ Observations:

- Mining association rules and mining frequent patterns are very close problems
- Scalable methods are needed for mining large datasets

(Q: Are these all the rules satisfying the two conditions?)

Challenge: There Are Too Many Frequent Patterns!

- A long pattern contains a combinatorial number of sub-patterns
- How many frequent itemsets does the following TDB₁ contain (minsup = 1)?

□ TDB₁: T₁: {a₁, ..., a₅₀}; T₂: {a₁, ..., a₁₀₀}

□ Let's have a try

1-itemsets: {a₁}: 2, {a₂}: 2, ..., {a₅₀}: 2, {a₅₁}: 1, ..., {a₁₀₀}: 1,

2-itemsets: {a₁, a₂}: 2, ..., {a₁, a₅₀}: 2, {a₁, a₅₁}: 1 ..., ..., {a₉₉, a₁₀₀}: 1,

..., ..., ..., ...

99-itemsets: {a₁, a₂, ..., a₉₉}: 1, ..., {a₂, a₃, ..., a₁₀₀}: 1

100-itemset: {a₁, a₂, ..., a₁₀₀}: 1

- The total number of frequent itemsets:

$$\binom{100}{1} + \binom{100}{2} + \binom{100}{3} + \cdots + \binom{100}{100} = 2^{100} - 1$$

A too huge set for any one to compute or store!



Expressing Patterns in Compressed Form: Closed Patterns

- ❑ How to handle such a challenge?
- ❑ Solution 1: **Closed patterns**: A pattern (itemset) X is **closed** if X is *frequent*, and there exists *no super-pattern* $Y \supset X$, with the same support as X
 - ❑ Let Transaction DB TDB_1 : $T_1: \{a_1, \dots, a_{50}\}$; $T_2: \{a_1, \dots, a_{100}\}$
 - ❑ Suppose $minsup = 1$. How many closed patterns does TDB_1 contain?
 - ❑ Two: $P_1: \{\{a_1, \dots, a_{50}\}: 2\}$; $P_2: \{\{a_1, \dots, a_{100}\}: 1\}$
- ❑ **Closed pattern** is a **lossless compression** of frequent patterns
 - ❑ Reduces the # of patterns but does not lose the support information!
 - ❑ You will still be able to say: $\{\{a_2, \dots, a_{40}\}: 2\}$, $\{\{a_5, a_{51}\}: 1\}$

Expressing Patterns in Compressed Form: Max-Patterns

- ❑ Solution 2: **Max-patterns**: A pattern X is a **max-pattern** if X is frequent and there exists no frequent super-pattern $Y \supset X$
- ❑ Difference from close-patterns?
 - ❑ Do not care the real support of the sub-patterns of a max-pattern
 - ❑ Let Transaction DB TDB_1 : $T_1: \{a_1, \dots, a_{50}\}$; $T_2: \{a_1, \dots, a_{100}\}$
 - ❑ Suppose $minsup = 1$. How many max-patterns does TDB_1 contain?
 - ❑ One: $P: \{\{a_1, \dots, a_{100}\}: 1\}$
- ❑ **Max-pattern** is a **lossy compression**!
 - ❑ We only know $\{a_1, \dots, a_{40}\}$ is frequent
 - ❑ But we do not know the real support of $\{a_1, \dots, a_{40}\}$, ..., any more!
- ❑ Thus in many applications, mining close-patterns is more desirable than mining max-patterns

Pattern Mining: Basic Concepts and Methods

- ❑ Basic Concepts
- ❑ Frequent Itemset Mining Methods 
- ❑ Which Patterns Are Interesting?—Pattern Evaluation Methods
- ❑ Summary

Efficient Pattern Mining Methods

- ❑ The Downward Closure Property of Frequent Patterns
 - ❑ The Apriori Algorithm
 - ❑ Extensions or Improvements of Apriori
- ❑ Mining Frequent Patterns by Exploring Vertical Data Format
- ❑ FPGrowth: A Frequent Pattern-Growth Approach
- ❑ Mining Closed Patterns

The Downward Closure Property of Frequent Patterns

- ❑ Observation: From TDB₁: $T_1: \{a_1, \dots, a_{50}\}$; $T_2: \{a_1, \dots, a_{100}\}$
 - ❑ We get a frequent itemset: $\{a_1, \dots, a_{50}\}$
 - ❑ Also, its subsets are all frequent: $\{a_1\}, \{a_2\}, \dots, \{a_{50}\}, \{a_1, a_2\}, \dots, \{a_1, \dots, a_{49}\}, \dots$
 - ❑ There are some hidden relationships among frequent patterns!
- ❑ The **downward closure (also called “Apriori”)** property of frequent patterns
 - ❑ If **{beer, diaper, nuts}** is frequent, so is **{beer, diaper}**
 - ❑ Every transaction containing {beer, diaper, nuts} also contains {beer, diaper}
 - ❑ Apriori: Any subset of a frequent itemset must be frequent
- ❑ Efficient mining methodology
 - ❑ If **any subset of an itemset S** is infrequent, then there is no chance for S to be frequent—why do we even have to consider S!?  A sharp knife for pruning!

Apriori Pruning and Scalable Mining Methods

- ❑ Apriori pruning principle: If there is any itemset which is infrequent, its superset should not even be generated! (Agrawal & Srikant @VLDB'94, Mannila, et al. @ KDD' 94)
- ❑ Scalable mining Methods: Three major approaches
 - ❑ Level-wise, join-based approach: Apriori (Agrawal & Srikant@VLDB'94)
 - ❑ Vertical data format approach: Eclat (Zaki, Parthasarathy, Ogihara, Li @KDD'97)
 - ❑ Frequent pattern projection and growth: FPgrowth (Han, Pei, Yin @SIGMOD'00)

Apriori: A Candidate Generation & Test Approach

- ❑ Outline of Apriori (level-wise, candidate generation and test)
 - ❑ Scan DB once to get frequent 1-itemset
 - ❑ Repeat
 - ❑ Generate length-(k+1) candidate itemsets from length-k frequent itemsets
 - ❑ Test the candidates against DB to find frequent (k+1)-itemsets
 - ❑ Set $k := k + 1$
 - ❑ Until no frequent or candidate set can be generated
 - ❑ Return all the frequent itemsets derived

The Apriori Algorithm (Pseudo-Code)

C_k : Candidate itemset of size k

F_k : Frequent itemset of size k

$K := 1$;

$F_k := \{\text{frequent items}\}$; // frequent 1-itemset

While ($F_k \neq \emptyset$) **do** { // when F_k is non-empty

$C_{k+1} := \text{candidates generated from } F_k$; // candidate generation

 Derive F_{k+1} by counting candidates in C_{k+1} with respect to TDB at minsup;

$k := k + 1$

}

return $\cup_k F_k$ // return F_k generated at each level

The Apriori Algorithm—An Example

số mặt hàng = 5

tính từng tập con -1 phần

vòng lặp 1

Database TDB

minsup = 2

Tid	Items
10	A, C, D
20	B, C, E
30	A, B, C, E
40	B, E

1st scan

C_1

Itemset	sup
{A}	2
{B}	3
{C}	3
{D}	1
{E}	3

F_1

không thỏa
minsup

Itemset	sup
{A}	2
{B}	3
{C}	3
{E}	3

vòng lặp 2: kết hợp các phần tử với nhau

C_2

Itemset	sup
{A, B}	1
{A, C}	2
{A, E}	1
{B, C}	2
{B, E}	3
{C, E}	2

2nd scan

C_2

Itemset
{A, B}
{A, C}
{A, E}
{B, C}
{B, E}
{C, E}

F_2

Itemset	sup
{A, C}	2
{B, C}	2
{B, E}	3
{C, E}	2

{A,B,C} => không thỏa vì {A,B} đã không t
tương tự với các tập không thỏa còn lại
vì thế chỉ xét 1 tập cuối cùng

C_3

Itemset
{B, C, E}

3rd scan

F_3

Itemset	sup
{B, C, E}	2

tổng cộng 9 tập phổ biến

Apriori: Implementation Tricks

□ How to generate candidates?

□ Step 1: self-joining F_k

□ Step 2: pruning

□ Example of candidate-generation

□ $F_3 = \{abc, abd, acd, ace, bcd\}$

□ Self-joining: $F_3 * F_3$

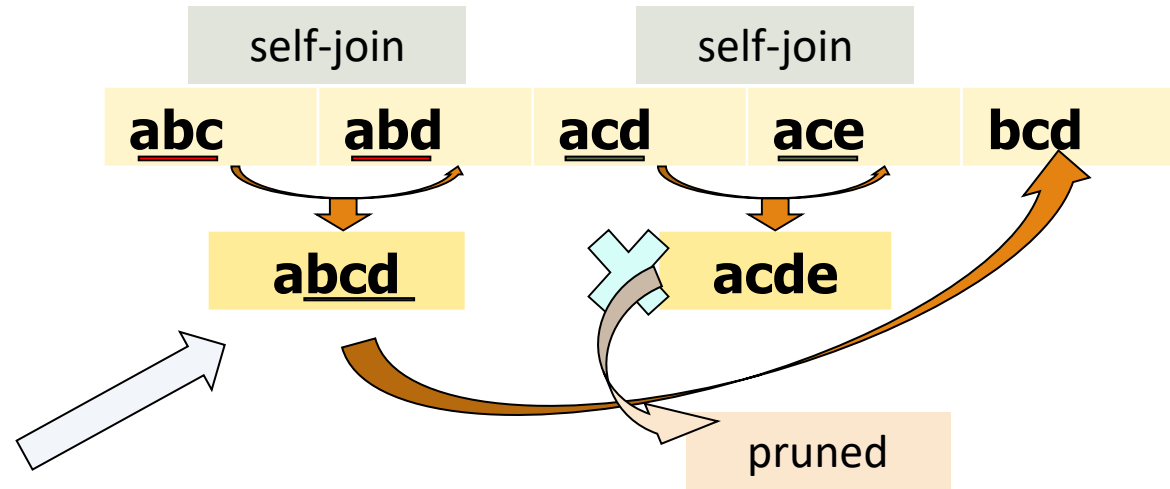
□ $abcd$ from abc and abd

□ $acde$ from acd and ace

□ Pruning:

□ $acde$ is removed because ade is not in F_3

□ $C_4 = \{abcd\}$



Candidate Generation (Pseudo-Code)

□ Suppose the items in F_{k-1} are listed in an order

□ // Step 1: Joining

for each p in F_{k-1}

for each q in F_{k-1}

if $p.item_1 = q.item_1, \dots, p.item_{k-2} = q.item_{k-2}, p.item_{k-1} < q.item_{k-1}$ {

$c = \text{join}(p, q)$

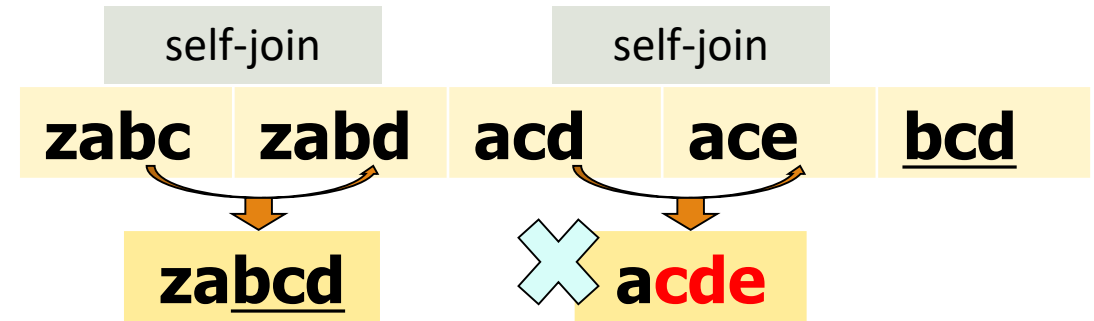
□ // Step 2: pruning

if $\text{has_infrequent_subset}(c, F_{k-1})$

continue // prune

else add c to C_k

}



Apriori: Improvements and Alternatives

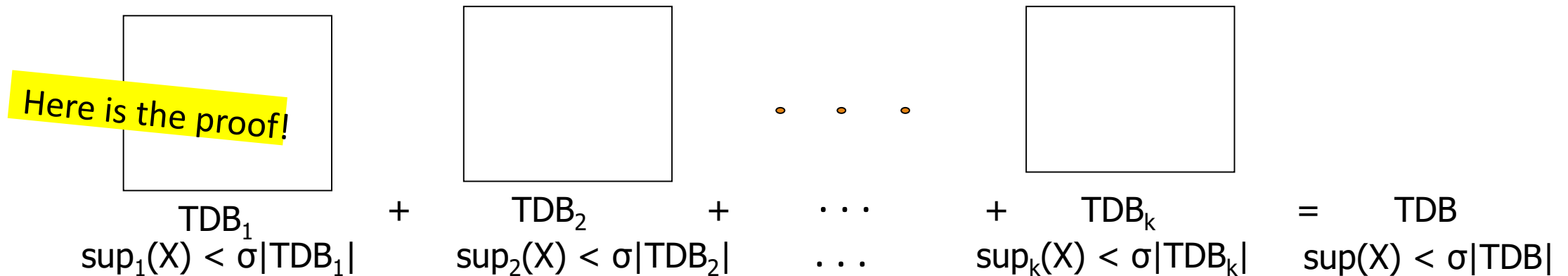
- ❑ Reduce passes of transaction database scans
 - ❑ **Partitioning** (e.g., Savasere, et al., 1995) 

To be discussed in subsequent slides
 - ❑ Dynamic itemset counting (Brin, et al., 1997)
- ❑ Shrink the number of candidates
 - ❑ **Hashing** (e.g., DHP: Park, et al., 1995) 

To be discussed in subsequent slides
 - ❑ Pruning by support lower bounding (e.g., Bayardo 1998)
 - ❑ Sampling (e.g., Toivonen, 1996)
- ❑ Exploring special data structures
 - ❑ Tree projection (Agarwal, et al., 2001)
 - ❑ H-miner (Pei, et al., 2001)
 - ❑ Hypercube decomposition (e.g., LCM: Uno, et al., 2004)

Partitioning: Scan Database Only Twice

- Theorem: *Any itemset that is potentially frequent in TDB must be frequent in at least one of the partitions of TDB*



- Method: Scan DB twice (A. Savasere, E. Omiecinski and S. Navathe, *VLDB'95*)
 - Scan 1: Partition database so that each partition can fit in main memory (why?)
 - Mine local frequent patterns in this partition
 - Scan 2: Consolidate global frequent patterns
 - Find global frequent itemset candidates (those frequent in at least one partition)
 - Find the true frequency of those candidates, by scanning TDB_i one more time

Direct Hashing and Pruning (DHP)

- ❑ **Hashing:** $v = \text{hash}(\text{itemset})$
- ❑ **1st scan:** When counting the 1-itemset, hash 2-itemset to calculate the bucket count
- ❑ **Example:** At the 1st scan of TDB, count 1-itemset, and hash 2-itemsets in the transaction to its bucket

- ❑ {ab, ad, ce}
- ❑ {bd, be, de}
- ❑ ...

- ❑ At the end of the first scan,
 - ❑ if $\text{minsup} = 80$, remove *ab, ad, ce*, since $\text{count}\{ab, ad, ce\} < 80$

V might be same for different itemset

Itemsets	Count
{ab, ad, ce}	35
{bd, be, de}	298
.....	...
{yz, qs, wt}	58

Hash Table



Check the minsup

A key observation: A k -itemset cannot be frequent if its corresponding hashing bucket count is below the *minsup* threshold

DHP (Direct Hashing and Pruning): (J. Park, M. Chen, and P. Yu, SIGMOD'95)

Exploring Vertical Data Format: ECLAT

- ❑ ECLAT (Equivalence Class Transformation): A **depth-first search** algorithm using set intersection [Zaki et al. @KDD'97]
- ❑ Vertical format
- ❑ Properties of Tid-Lists
 - ❑ $t(X) = t(Y)$: X and Y always happen together (e.g., $t(ac) = t(d)$)
 - ❑ $t(X) \subset t(Y)$: transaction having X always has Y (e.g., $t(ac) \subset t(ce)$)
- ❑ Frequent patterns: vertical intersections
- ❑ Using **diffset** to accelerate mining
 - ❑ Only keep track of differences of tids
 - ❑ $t(e) = \{T_{10}, T_{20}, T_{30}\}$, $t(ce) = \{T_{10}, T_{30}\} \rightarrow \text{Diffset}(ce, e) = \{T_{20}\}$

A transaction DB in Horizontal Data Format

Tid	Itemset
10	a, c, d, e
20	a, b, e
30	b, c, e

The transaction DB in Vertical Data Format

Item	TidList
a	10, 20
b	20, 30
c	10, 30
d	10
e	10, 20, 30

$t(e) = \{T_{10}, T_{20}, T_{30}\};$
 $t(a) = \{T_{10}, T_{20}\};$
 $t(ae) = \{T_{10}, T_{20}\}$

Why Mining Frequent Patterns by Pattern Growth?

quét DB nhiều lần, đi theo chiều rộng => tìm một thuật toán khác theo chiều sâu -> FP (

- ❑ Apriori: A *breadth-first search* mining algorithm
 - ❑ First find the complete set of frequent k-itemsets
 - ❑ Then derive frequent (k+1)-itemset candidates
 - ❑ Scan DB again to find true frequent (k+1)-itemsets
- ❑ Motivation for a different mining methodology
 - ❑ Can we develop a *depth-first search* mining algorithm?
 - ❑ For a frequent itemset p , can subsequent search be confined to only those transactions that containing p ?
- ❑ Such thinking leads to a frequent pattern growth approach: FPGrowth

FPGrowth (J. Han, J. Pei, Y. Yin, “Mining Frequent Patterns without Candidate Generation,” SIGMOD 2000)

Example: From Transactional DB to Ordered Frequent Itemlist

Example: A Sample Transactional Database

TID	Items in the Transaction
100	{f, a, c, d, g, i, m, p}
200	{a, b, c, f, l, m, o}
300	{b, f, h, j, o, w}
400	{b, c, k, s, p}
500	{a, f, c, e, l, p, m, n}

Let min_support = 3

- Scan DB once, find single item frequent pattern: f:4, a:3, c:4, b:3, m:3, p:3
- Sort frequent items in frequency descending order, f-list F-list = f-c-a-b-m-p m xuất hiện trước p nên đi trước p
- Scan DB again, use the ordered frequent itemlist for each transaction to construct an

FP-tree

TID	Items in the Transaction	Ordered, frequent itemlist
100	{f, a, c, d, g, i, m, p}	f, c, a, m, p
200	{a, b, c, f, l, m, o}	f, c, a, b, m
300	{b, f, h, j, o, w}	f, b
400	{b, c, k, s, p}	c, b, p
500	{a, f, c, e, l, p, m, n}	f, c, a, m, p

tôn trọng thứ tự xuất hiệ

quét DB đúng 2 lần
Lần 1: tính count
Lần 2: xây dựng cây

Example: Construct FP-tree from Transaction DB

TID	Ordered, frequent itemlist
100	<i>f, c, a, m, p</i>
200	<i>f, c, a, b, m</i>
300	<i>f, b</i>
400	<i>c, b, p</i>
500	<i>f, c, a, m, p</i>

FP-Tree Construction:

For each transaction, insert the ordered frequent itemlist into an FP-tree, with shared sub-branches merged, counts accumulated

After inserting the 1st frequent Itemlist:
"f, c, a, m, p"

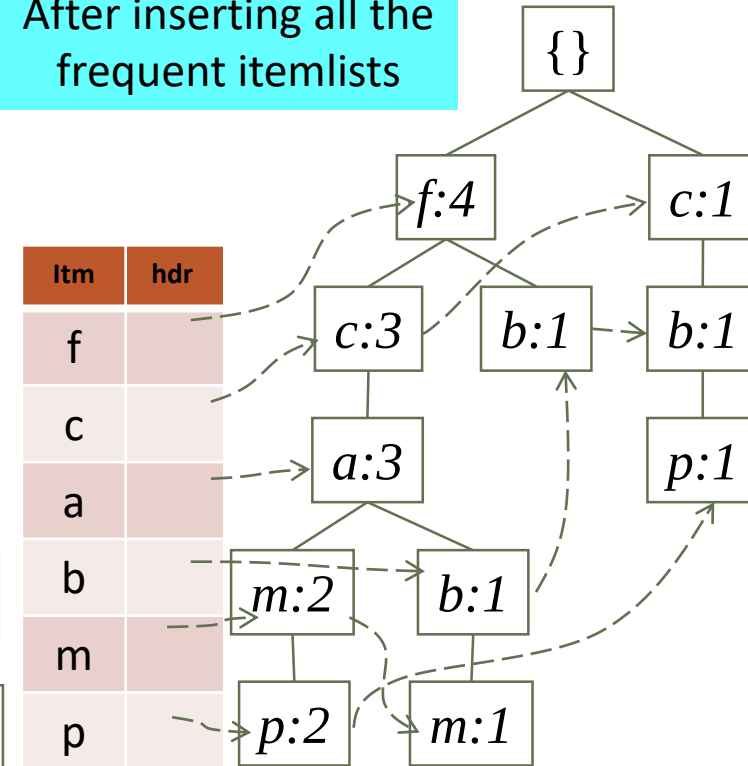
Item	Frqncy	hdr
f	4	
c	4	
a	3	
b	3	
m	3	
p	3	

Header Table

After inserting the 2nd frequent itemlist
"f, c, a, b, m"

Itm	hdr
f	
c	
a	
b	
m	
p	

After inserting all the frequent itemlists

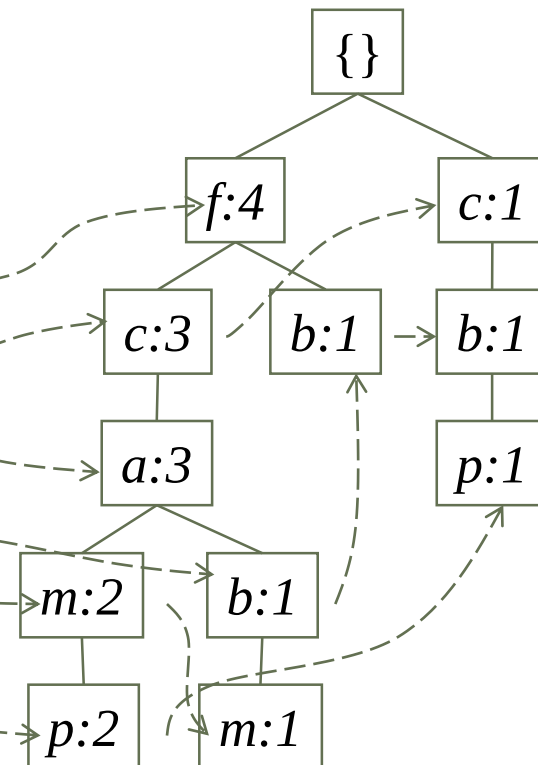


Mining FP-Tree: Divide and Conquer Based on Patterns and Data

- Pattern mining can be partitioned according to current patterns
 - Patterns containing p : p 's conditional database: $fcam:2, cb:1$
 - p 's conditional database (i.e., the database under the condition that p exists):
 - *transformed prefix paths* of item p
 - Patterns having m but no p : m 's conditional database: $fca:2, fcab:1$
 -

min_support = 3

Item	Frequency	Header
f	4	
c	4	
a	3	
b	3	
m	3	
p	3	



Conditional database of each pattern

<u>Item</u>	<u>Conditional database</u>
<i>c</i>	<i>f:3</i>
<i>a</i>	<i>fc:3</i>
<i>b</i>	<i>fca:1, f:1, c:1</i>
<i>m</i>	<i>fca:2, fcab:1</i>
<i>p</i>	<i>fcam:2, cb:1</i>

$p \rightarrow m \rightarrow a \rightarrow c \rightarrow f$, lấy min tất cả = 2
 $p \rightarrow b \rightarrow c$, lấy min tất cả = 1
 merge: $f:2, c:3, a:2, b:1, m:2 \Rightarrow$ mỗi c th
 $\Rightarrow$ tập phổ biến của p là $\{c, p\}$ và $\{p\}$

Mine Each Conditional Database Recursively

min_support = 3

Conditional Data Bases

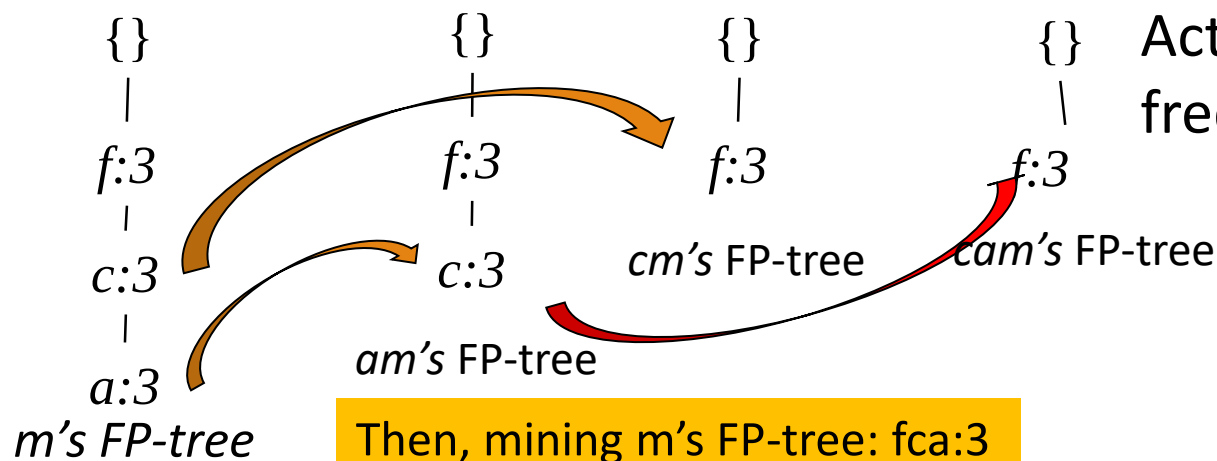
<i>item</i>	<i>cond. data base</i>
<i>c</i>	<i>f:3</i>
<i>a</i>	<i>fc:3</i>
<i>b</i>	<i>fca:1, f:1, c:1</i>
<i>m</i>	<i>fca:2, fcab:1</i>
<i>p</i>	<i>fcam:2, cb:1</i>

- For each conditional database
 - Mine single-item patterns
 - Construct its FP-tree & mine it

p's conditional DB: *fcam:2, cb:1* → *c: 3*

m's conditional DB: *fca:2, fcab:1* → *fca: 3*

b's conditional DB: *fca:1, f:1, c:1* → ϕ



Actually, for single branch FP-tree, all the frequent patterns can be generated in one shot

m: 3

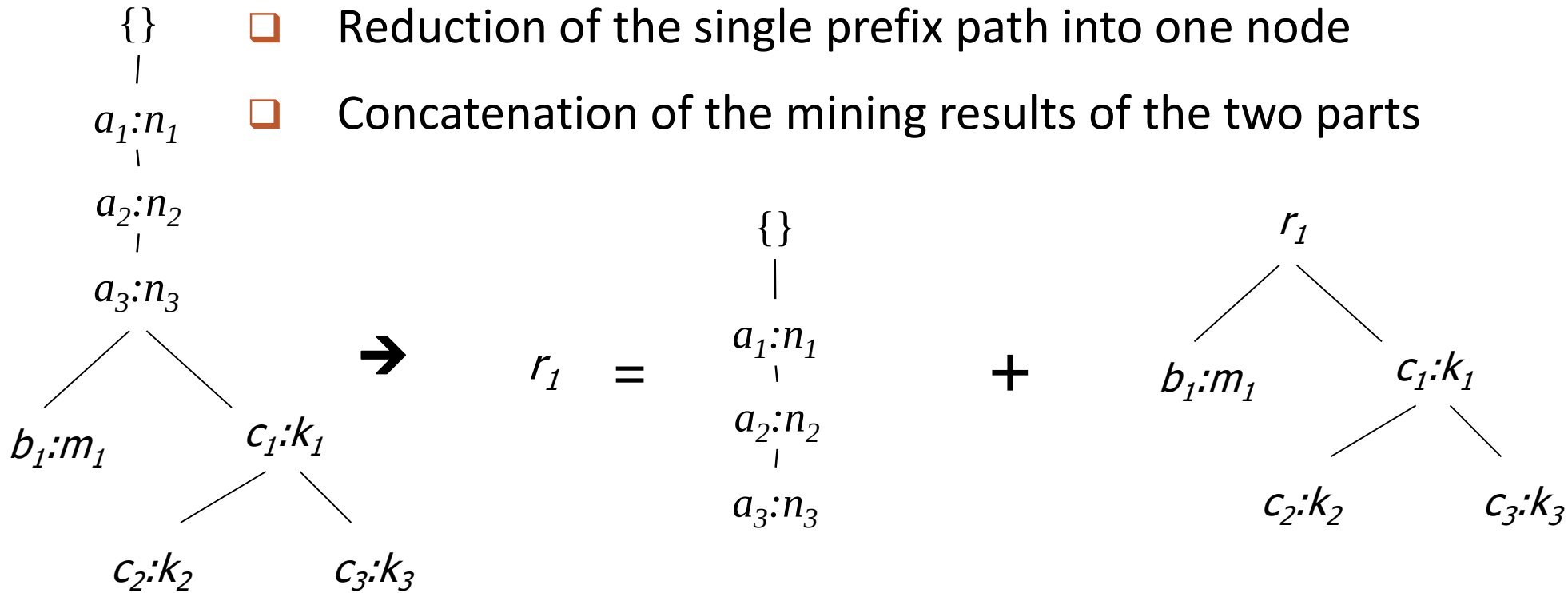
fm: 3, cm: 3, am: 3

fcm: 3, fam:3, cam: 3

fcam: 3

A Special Case: Single Prefix Path in FP-tree

- Suppose a (conditional) FP-tree T has a shared single prefix-path P
- Mining can be decomposed into two parts

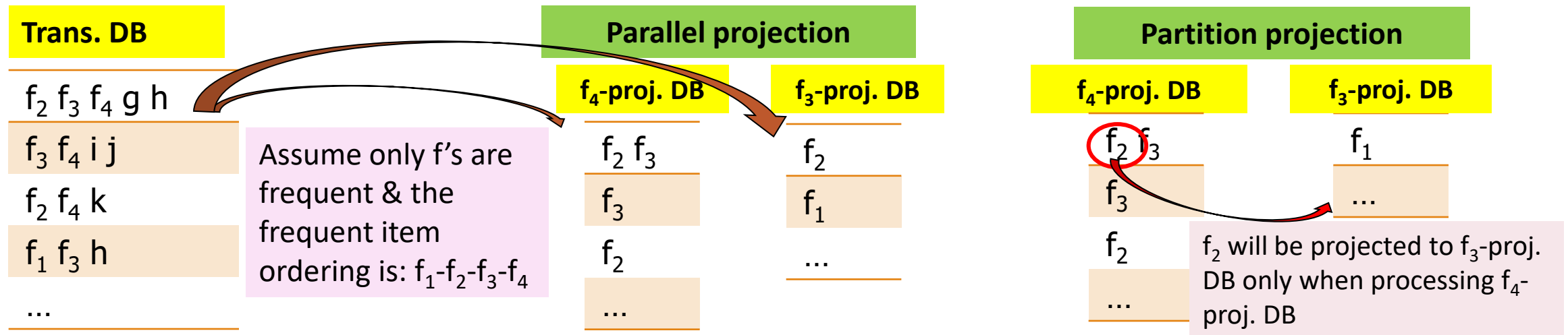


FPGrowth: Mining Frequent Patterns by Pattern Growth

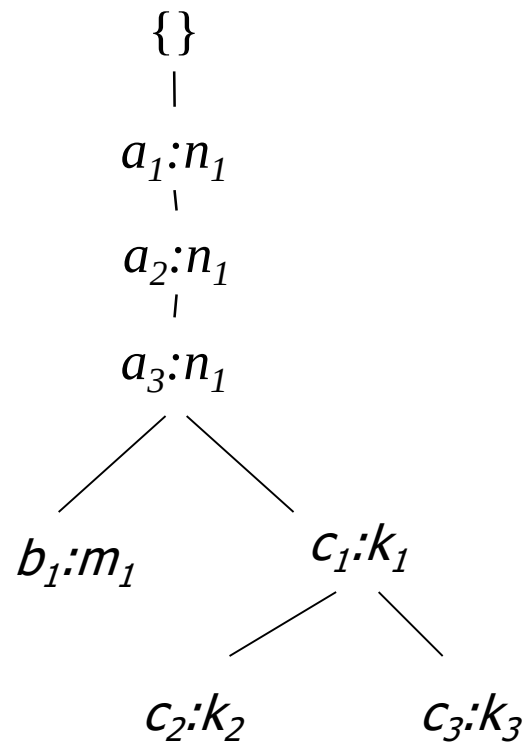
- ❑ Essence of frequent pattern growth (FPGrowth) methodology
 - ❑ Find frequent single items and partition the database based on each such single item pattern
 - ❑ Recursively grow frequent patterns by doing the above for each *partitioned database* (also called the pattern's *conditional database*)
 - ❑ To facilitate efficient processing, an efficient data structure, FP-tree, can be constructed
- ❑ Mining becomes
 - ❑ Recursively construct and mine (conditional) FP-trees
 - ❑ Until the resulting FP-tree is empty, or until it contains only one path—single path will generate all the combinations of its sub-paths, each of which is a frequent pattern

Scaling FP-growth by Item-Based Data Projection

- What if FP-tree cannot fit in memory?—Do not construct FP-tree
 - “Project” the database based on frequent single items
 - Construct & mine FP-tree for each projected DB
- Parallel projection** vs. **partition projection**
 - Parallel projection: Project the DB on each frequent item
 - Space costly, all partitions can be processed in parallel
 - Partition projection: Partition the DB in order
 - Passing the unprocessed parts to subsequent partitions



CLOSET+: Mining Closed Itemsets by Pattern-Growth



- ❑ Efficient, *direct* mining of closed itemsets
- ❑ Intuition:
 - ❑ If an FP-tree contains a single branch as shown left
 - ❑ “ a_1, a_2, a_3 ” should be merged
- ❑ Itemset merging: If Y appears in every occurrence of X, then Y is merged with X
 - ❑ d -proj. db: {acef, acf} $\rightarrow$ $acfd$ -proj. db: {e}
 - ❑ Final closed itemset: $acfd:2$
- ❑ There are many other tricks developed
 - ❑ For details, see J. Wang, et al., “CLOSET+: Searching for the Best Strategies for Mining Frequent Closed Itemsets”, KDD'03

TID	Items
1	acdef
2	abe
3	cefg
4	acdf

Let minsupport = 2

$a:3, c:3, d:2, e:3, f:3$

F-List: a-c-e-f-d

Pattern Mining: Basic Concepts and Methods

- ❑ Basic Concepts
- ❑ Frequent Itemset Mining Methods
- ❑ Which Patterns Are Interesting?—Pattern Evaluation Methods
- ❑ Summary



How to Judge if a Rule/Pattern Is Interesting?

- ❑ Pattern-mining will generate a large set of patterns/rules
 - ❑ Not all the generated patterns/rules are interesting
- ❑ Interestingness measures: Objective vs. subjective
 - ❑ Objective interestingness measures
 - ❑ Support, confidence, correlation, ...
 - ❑ Subjective interestingness measures:
 - ❑ Different users may judge interestingness differently
 - ❑ Let a user specify
 - ❑ Query-based: Relevant to a user's particular request
 - ❑ Judge against one's knowledge-base
 - ❑ unexpected, freshness, timeliness

Limitation of the Support-Confidence Framework

- ❑ Are s and c interesting in association rules: " $A \Rightarrow B$ " [s, c]? **Be careful!**
- ❑ Example: Suppose one school may have the following statistics on # of students who may play basketball and/or eat cereal:

	play-basketball	not play-basketball	sum (row)
eat-cereal	400	350	750
not eat-cereal	200	50	250
sum(col.)	600	400	1000

2-way contingency table

- ❑ Association rule mining may generate the following:
 - ❑ $play\text{-}basketball \Rightarrow eat\text{-}cereal$ [40%, 66.7%] (higher s & c)
- ❑ But this strong association rule is misleading: The overall % of students eating cereal is 75% > 66.7%, a more telling rule:
 - ❑ $\neg play\text{-}basketball \Rightarrow eat\text{-}cereal$ [35%, 87.5%] (high s & c)

Interestingness Measure: Lift

- Measure of dependent/correlated events: **lift**

$$lift(B, C) = \frac{c(B \rightarrow C)}{s(C)} = \frac{s(B \cup C)}{s(B) \times s(C)}$$

Lift is more telling than s & c

	B	¬B	Σ _{row}
C	400	350	750
¬C	200	50	250
Σ _{col.}	600	400	1000

- Lift(B, C) may tell how B and C are correlated

- Lift(B, C) = 1: B and C are independent
 - > 1: positively correlated
 - < 1: negatively correlated

- For our example,

$$lift(B, C) = \frac{400 / 1000}{600 / 1000 \times 750 / 1000} = 0.89$$

$$lift(B, \neg C) = \frac{200 / 1000}{600 / 1000 \times 250 / 1000} = 1.33$$

- Thus, B and C are negatively correlated since $lift(B, C) < 1$;
 - B and ¬C are positively correlated since $lift(B, \neg C) > 1$

Interestingness Measure: χ^2

- Another measure to test correlated events: χ^2

$$\chi^2 = \sum \frac{(\text{Observed} - \text{Expected})^2}{\text{Expected}}$$

- For the table on the right,

$$\chi^2 = \frac{(400 - 450)^2}{450} + \frac{(350 - 300)^2}{300} + \frac{(200 - 150)^2}{150} + \frac{(50 - 100)^2}{100} = 55.56$$

- Lookup χ^2 distribution table $\rightarrow$ B, C are correlated
- χ^2 -test shows B and C are negatively correlated since the expected value is 450 but the observed is only 400
- Thus, χ^2 is also more telling than the support-confidence framework

	B	$\neg B$	Σ_{row}
C	400 (450)	350 (300)	750
$\neg C$	200 (150)	50 (100)	250
Σ_{col}	600	400	1000

Expected value

Observed value

Lift and χ^2 : Are They Always Good Measures?

- ❑ Null transactions: Transactions that contain neither B nor C
- ❑ Let's examine the new dataset D
 - ❑ BC (100) is much rarer than B¬C (1000) and ¬BC (1000), but there are many ¬B¬C (100000)
 - ❑ Unlikely B & C will happen together!
- ❑ But, $\text{Lift}(B, C) = 8.44 \gg 1$ (Lift shows B and C are strongly positively correlated!)
- ❑ $\chi^2 = 670$: Observed(BC) $\gg$ expected value (11.85)
- ❑ Too many *null transactions* may “spoil the soup”!



	B	¬B	Σ_{row}
C	100	1000	1100
¬C	1000	100000	101000
$\Sigma_{\text{col.}}$	1100	101000	102100

null transactions

Contingency table with expected values added

	B	¬B	Σ_{row}
C	100 (11.85)	1000	1100
¬C	1000 (988.15)	100000	101000
$\Sigma_{\text{col.}}$	1100	101000	102100

Interestingness Measures & Null-Invariance

- **Null invariance** means: The number of null transactions does not matter.
Does not change the measure value.
- A few interestingness measures: Some are null invariant

Measure	Definition	Range	Null-Invariant?
$\chi^2(A, B)$	$\sum_{i,j} \frac{(e(a_i, b_j) - o(a_i, b_j))^2}{e(a_i, b_j)}$	$[0, \infty]$	No
$Lift(A, B)$	$\frac{s(A \cup B)}{s(A) \times s(B)}$	$[0, \infty]$	No
$Allconf(A, B)$	$\frac{s(A \cup B)}{\max\{s(A), s(B)\}}$	$[0, 1]$	Yes
$Jaccard(A, B)$	$\frac{s(A \cup B)}{s(A) + s(B) - s(A \cup B)}$	$[0, 1]$	Yes
$Cosine(A, B)$	$\frac{s(A \cup B)}{\sqrt{s(A) \times s(B)}}$	$[0, 1]$	Yes
$Kulczynski(A, B)$	$\frac{1}{2} \left(\frac{s(A \cup B)}{s(A)} + \frac{s(A \cup B)}{s(B)} \right)$	$[0, 1]$	Yes
$MaxConf(A, B)$	$\max\left\{ \frac{s(A \cup B)}{s(A)}, \frac{s(A \cup B)}{s(B)} \right\}$	$[0, 1]$	Yes

Let

$$p = \frac{s(A \cup B)}{s(A)} = P(B|A)$$

$$q = \frac{s(A \cup B)}{s(B)} = P(A|B)$$

p, q are null invariant

Essentially min,
max, mean variants
of p, q

Null Invariance: An Important Property

- Why is null invariance crucial for the analysis of massive transaction data?
 - Many transactions may contain neither milk nor coffee!

milk vs. coffee contingency table

	<i>milk</i>	$\neg milk$	Σ_{row}
<i>coffee</i>	<i>mc</i>	$\neg mc$	<i>c</i>
$\neg coffee$	<i>m</i> $\neg c$	$\neg m$ $\neg c$	$\neg c$
Σ_{col}	<i>m</i>	$\neg m$	Σ

- Lift and χ^2 are not null-invariant: not good to evaluate data that contain too many or too few null transactions!
- Many measures are not null-invariant!

Null-transactions
w.r.t. *m* and *c*

Data set	<i>mc</i>	$\neg mc$	<i>m</i> $\neg c$	$\neg m$ $\neg c$	χ^2	<i>Lift</i>
<i>D</i> ₁	10,000	1,000	1,000	100,000	90557	9.26
<i>D</i> ₂	10,000	1,000	1,000	100	0	1
<i>D</i> ₃	100	1,000	1,000	100,000	670	8.44
<i>D</i> ₄	1,000	1,000	1,000	100,000	24740	25.75
<i>D</i> ₅	1,000	100	10,000	100,000	8173	9.18
<i>D</i> ₆	1,000	10	100,000	100,000	965	1.97

Comparison of Null-Invariant Measures

- ❑ Not all null-invariant measures are created equal
- ❑ Which one is better?
 - ❑ D_4 — D_6 differentiate the null-invariant measures
 - ❑ Kulc (Kulczynski 1927) holds firm and is in balance of both directional implications

2-variable contingency table

	<i>milk</i>	$\neg milk$	Σ_{row}
<i>coffee</i>	<i>mc</i>	$\neg mc$	<i>c</i>
$\neg coffee$	<i>m</i> $\neg c$	$\neg m \neg c$	$\neg c$
Σ_{col}	<i>m</i>	$\neg m$	Σ

All 5 are null-invariant

Data set	<i>mc</i>	$\neg mc$	<i>m</i> $\neg c$	$\neg m \neg c$	<i>AllConf</i>	Jaccard	<i>Cosine</i>	<i>Kulc</i>	<i>MaxConf</i>
D_1	10,000	1,000	1,000	100,000	0.91	0.83	0.91	0.91	0.91
D_2	10,000	1,000	1,000	100	0.91	0.83	0.91	0.91	0.91
D_3	100	1,000	1,000	100,000	0.09	0.05	0.09	0.09	0.09
D_4	1,000	1,000	1,000	100,000	0.5	0.33	0.5	0.5	0.5
D_5	1,000	100	10,000	100,000	0.09	0.09	0.29	0.5	0.91
D_6	1,000	10	100,000	100,000	0.01	0.01	0.10	0.5	0.99

Subtle: They disagree on those cases

Imbalance Ratio with Kulczynski Measure

- IR (Imbalance Ratio): measure the imbalance of two itemsets A and B in rule implications:

$$IR(A, B) = \frac{|s(A) - s(B)|}{s(A) + s(B) - s(A \cup B)}$$

- Kulczynski and Imbalance Ratio (IR) together present a clear picture for all the three datasets D_4 through D_6
 - D_4 is neutral & balanced; D_5 is neutral but imbalanced
 - D_6 is neutral but very imbalanced

Data set	mc	$\neg mc$	$m\neg c$	$\neg m\neg c$	Jaccard	Cosine	Kulc	IR
D_1	10,000	1,000	1,000	100,000	0.83	0.91	0.91	0
D_2	10,000	1,000	1,000	100	0.83	0.91	0.91	0
D_3	100	1,000	1,000	100,000	0.05	0.09	0.09	0
D_4	1,000	1,000	1,000	100,000	0.33	0.5	0.5	0
D_5	1,000	100	10,000	100,000	0.09	0.29	0.5	0.89
D_6	1,000	10	100,000	100,000	0.01	0.10	0.5	0.99

Example: Analysis of DBLP Coauthor Relationships

- DBLP: Computer science research publication bibliographic database
 - > 3.8 million entries on authors, paper, venue, year, and other information

ID	Author <i>A</i>	Author <i>B</i>	$s(A \cup B)$	$s(A)$	$s(B)$	Jaccard	<i>Cosine</i>	<i>Kulc</i>
1	Hans-Peter Kriegel	Martin Ester	28	146	54	0.163 (2)	0.315 (7)	0.355 (9)
2	Michael Carey	Miron Livny	26	104	58	0.191 (1)	0.335 (4)	0.349 (10)
3	Hans-Peter Kriegel	Joerg Sander	24	146	36	0.152 (3)	0.331 (5)	0.416 (8)
4	Christos Faloutsos	Spiros Papadimitriou	20	162	26	0.119 (7)	0.308 (10)	0.446 (7)
5	Hans-Peter Kriegel	Martin Pfeifle	18	146	18	0.123 (6)	0.351 (2)	0.562 (2)
6	Hector Garcia-Molina	Wilburt Labio	16	144	18	0.110 (9)	0.314 (8)	0.500 (4)
7	Divyakant Agrawal	Wang Hsiung	16	120	16	0.133 (5)	0.365 (1)	0.567 (1)
8	Elke Rundensteiner	Murali Mani	16	104	20	0.148 (4)	0.351 (3)	0.477 (6)
9	Divyakant Agrawal	Oliver Po	12	120	12	0.100 (10)	0.316 (6)	0.550 (3)
10	Gerhard Weikum	Martin Theobald	12	106	14	0.111 (8)	0.312 (9)	0.485 (5)

Advisor-advisee relation: Kulc: high, Jaccard: low, cosine: middle

- Which pairs of authors are strongly related? Is *A* the advisor, or the advisee?
 - Use Kulc to find Advisor-advisee, close collaborators

What Measures to Choose for Effective Pattern Evaluation?

- ❑ Null value cases are predominant in many large datasets
 - ❑ Neither milk nor coffee is in most of the baskets; neither Mike nor Jim is an author in most of the papers;
- ❑ *Null-invariance* is an important property
- ❑ Lift, χ^2 and cosine are good measures if null transactions are not predominant
 - ❑ Otherwise, *Kulczynski + Imbalance Ratio* should be used to judge the interestingness of a pattern
- ❑ Exercise: Mining research collaborations from research bibliographic data
 - ❑ Find a group of frequent collaborators from research bibliographic data (e.g., DBLP)
 - ❑ Can you find the likely advisor-advisee relationship and during which years such a relationship happened?
 - ❑ Ref.: C. Wang, J. Han, Y. Jia, J. Tang, D. Zhang, Y. Yu, and J. Guo, "Mining Advisor-Advisee Relationships from Research Publication Networks", KDD'10

Pattern Mining: Basic Concepts and Methods

- ❑ Basic Concepts
- ❑ Frequent Itemset Mining Methods
- ❑ Which Patterns Are Interesting?—Pattern Evaluation Methods
- ❑ Summary 

Summary

- ❑ Basic Concepts
 - ❑ What Is Pattern Discovery? Why Is It Important?
 - ❑ Basic Concepts: Frequent Patterns and Association Rules
 - ❑ Compressed Representation: Closed Patterns and Max-Patterns
- ❑ Efficient Pattern Mining Methods
 - ❑ The Downward Closure Property of Frequent Patterns
 - ❑ The Apriori Algorithm
 - ❑ Extensions or Improvements of Apriori
 - ❑ Mining Frequent Patterns by Exploring Vertical Data Format
 - ❑ FPGrowth: A Frequent Pattern-Growth Approach
 - ❑ Mining Closed Patterns
- ❑ Pattern Evaluation
 - ❑ Interestingness Measures in Pattern Mining
 - ❑ Interestingness Measures: Lift and χ^2
 - ❑ Null-Invariant Measures
 - ❑ Comparison of Interestingness Measures

Recommended Readings (Basic Concepts)

- ❑ R. Agrawal, T. Imielinski, and A. Swami, “Mining association rules between sets of items in large databases”, in Proc. of SIGMOD'93
- ❑ R. J. Bayardo, “Efficiently mining long patterns from databases”, in Proc. of SIGMOD'98
- ❑ N. Pasquier, Y. Bastide, R. Taouil, and L. Lakhal, “Discovering frequent closed itemsets for association rules”, in Proc. of ICDT'99
- ❑ J. Han, H. Cheng, D. Xin, and X. Yan, “Frequent Pattern Mining: Current Status and Future Directions”, Data Mining and Knowledge Discovery, 15(1): 55-86, 2007

Recommended Readings (Efficient Pattern Mining Methods)

- ❑ R. Agrawal and R. Srikant, “Fast algorithms for mining association rules”, VLDB'94
- ❑ A. Savasere, E. Omiecinski, and S. Navathe, “An efficient algorithm for mining association rules in large databases”, VLDB'95
- ❑ J. S. Park, M. S. Chen, and P. S. Yu, “An effective hash-based algorithm for mining association rules”, SIGMOD'95
- ❑ S. Sarawagi, S. Thomas, and R. Agrawal, “Integrating association rule mining with relational database systems: Alternatives and implications”, SIGMOD'98
- ❑ M. J. Zaki, S. Parthasarathy, M. Ogihara, and W. Li, “Parallel algorithm for discovery of association rules”, Data Mining and Knowledge Discovery, 1997
- ❑ J. Han, J. Pei, and Y. Yin, “Mining frequent patterns without candidate generation”, SIGMOD'00
- ❑ M. J. Zaki and Hsiao, “CHARM: An Efficient Algorithm for Closed Itemset Mining”, SDM'02
- ❑ J. Wang, J. Han, and J. Pei, “CLOSET+: Searching for the Best Strategies for Mining Frequent Closed Itemsets”, KDD'03
- ❑ C. C. Aggarwal, M.A., Bhuiyan, M. A. Hasan, “Frequent Pattern Mining Algorithms: A Survey”, in Aggarwal and Han (eds.): Frequent Pattern Mining, Springer, 2014

Recommended Readings (Pattern Evaluation)

- ❑ C. C. Aggarwal and P. S. Yu. A New Framework for Itemset Generation. PODS'98
- ❑ S. Brin, R. Motwani, and C. Silverstein. Beyond market basket: Generalizing association rules to correlations. SIGMOD'97
- ❑ M. Klemettinen, H. Mannila, P. Ronkainen, H. Toivonen, and A. I. Verkamo. Finding interesting rules from large sets of discovered association rules. CIKM'94
- ❑ E. Omiecinski. Alternative Interest Measures for Mining Associations. TKDE'03
- ❑ P.-N. Tan, V. Kumar, and J. Srivastava. Selecting the Right Interestingness Measure for Association Patterns. KDD'02
- ❑ T. Wu, Y. Chen and J. Han, Re-Examination of Interestingness Measures in Pattern Mining: A Unified Framework, Data Mining and Knowledge Discovery, 21(3):371-397, 2010

Pattern Mining: Advanced Methods

- ☐ Mining Various Kinds of Patterns 
- ☐ Mining Compressed or Approximate Patterns
- ☐ Constraint-based Pattern Mining
- ☐ Mining Sequential Patterns
- ☐ Mining Subgraph Patterns
- ☐ Pattern Mining: Application Examples
- ☐ Summary

Mining Various Kinds of Patterns

- ❑ Mining Multilevel Associations
- ❑ Mining Multidimensional Associations
- ❑ Mining Quantitative Association Rules
- ❑ Mining High-dimensional Data
- ❑ Mining Rare Patterns and Negative Patterns

Mining Multiple-Level Frequent Patterns

- Items often form hierarchies

- Milk: Dairyland 2% milk
- Bread: Wonder wheat bread

Uniform support

Level 1
min_sup = 5%

Level 2
min_sup = 5%

Milk
[support = 10%]

2% Milk
[support = 6%]

Skim Milk
[support = 2%]

Reduced support

Level 1
min_sup = 5%

Level 2
min_sup = 1%

- Min-support thresholds for items following concept hierarchies
 - **Uniform** min-support across multiple levels (reasonable?)
 - **Level-reduced** min-support:
 - Items at the lower level are expected to have lower support
- Efficient mining: *Shared* multi-level mining
 - Use the lowest min-support to pass down the set of candidates

Redundancy Filtering at Mining Multi-Level Associations

- ❑ Redundancy filtering: redundant due to “ancestor” relationships
 - ❑ milk $\Rightarrow$ wheat bread [support = 8%, confidence = 70%] (1)
 - ❑ 2% milk $\Rightarrow$ wheat bread [support = 2%, confidence = 72%] (2)
 - ❑ Suppose the 2% milk sold is about $\frac{1}{4}$ of milk sold in gallons
 - ❑ (2) should be able to be “derived” from (1)
- ❑ **In general:** A rule is *redundant* if its support is close to the “expected” value, according to its “ancestor” rule, and it has a similar confidence as its “ancestor”

Q: If rule (1) is an ancestor of rule (2), which one to prune?

Customized Min-Supports for Different Kinds of Items

- ❑ We have used the same min-support threshold for all the items or item sets to be mined in each association mining
- ❑ In reality, some items (e.g., diamond, watch, ...) are valuable but less frequent
- ❑ It is necessary to have customized min-support settings for different kinds of items
- ❑ One Method: Use **group-based “individualized” min-support**
 - ❑ E.g., {diamond, watch}: 0.05%; {bread, milk}: 5%; ...
 - ❑ How to mine such rules efficiently?
 - ❑ Existing scalable mining algorithms can be easily extended to cover such cases

Mining Multi-Dimensional Associations

- ❑ **Single-dimensional** rules (e.g., items are all in “product” dimension)
 - ❑ $\text{buys}(X, \text{“milk”}) \Rightarrow \text{buys}(X, \text{“bread”})$
- ❑ **Multi-dimensional** rules (i.e., items in ≥ 2 dimensions or predicates)
 - ❑ Inter-dimension association rules (*no repeated predicates*)
 - ❑ $\text{age}(X, \text{“18-25”}) \wedge \text{occupation}(X, \text{“student”}) \Rightarrow \text{buys}(X, \text{“coke”})$
 - ❑ Hybrid-dimension association rules (*repeated predicates*)
 - ❑ $\text{age}(X, \text{“18-25”}) \wedge \text{buys}(X, \text{“popcorn”}) \Rightarrow \text{buys}(X, \text{“coke”})$
- ❑ Attributes can be categorical or numerical
 - ❑ Categorical Attributes (e.g., *profession*, *product*: no ordering among values): Data cube for inter-dimension association
 - ❑ Quantitative Attributes: Numeric, implicit ordering among values—discretization, clustering, and gradient approaches

Mining Quantitative Associations

- ❑ Mining associations with numerical attributes
 - ❑ E.g.: Numerical attributes: **age** and **salary**
- ❑ Methods
 - ❑ **Static discretization** based on predefined concept hierarchies
 - ❑ Discretization on each dimension with hierarchy
 - ❑ age: {0-10, 10-20, ..., 90-100} → {young, mid-aged, old}
 - ❑ **Dynamic discretization** based on data distribution
 - ❑ **Clustering**: Distance-based association
 - ❑ First one-dimensional clustering, then association
 - ❑ **Deviation analysis**:
 - ❑ Gender = female $\Rightarrow$ Wage: mean=\$7/hr (overall mean = \$9)

Mining Extraordinary Phenomena in Quantitative Association Mining

- ❑ Mining extraordinary (i.e., interesting) phenomena
 - ❑ E.g.: **Gender = female** $\Rightarrow$ **Wage: mean=\$7/hr** (overall mean = \$9)
 - ❑ **LHS**: a subset of the population
 - ❑ **RHS**: an extraordinary behavior of this subset
- ❑ The rule is accepted only if a statistical test (e.g., Z-test) confirms the inference with high confidence
- ❑ Subrule: Highlights the extraordinary behavior of a subset of the population of the super rule
 - ❑ E.g.: **(Gender = female) ^ (South = yes) $\Rightarrow$ mean wage = \$6.3/hr**
- ❑ Rule condition can be categorical or numerical (quantitative rules)
 - ❑ E.g.: **Education in [14-18] (yrs) $\Rightarrow$ mean wage = \$11.64/hr**

Efficient methods have been developed for mining such rules (e.g., Aumann and Lindell@KDD'99)

Rare Patterns vs. Negative Patterns

❑ Rare patterns

- ❑ Very low support but interesting (e.g., buying Rolex watches)
- ❑ How to mine them? Setting individualized, group-based min-support thresholds for different groups of items

❑ Negative patterns

- ❑ Negatively correlated: Unlikely to happen together
- ❑ Ex.: Since it is unlikely that the same customer buys both a **Ford Expedition** (an SUV car) and a **Ford Fusion** (a hybrid car), buying a **Ford Expedition** and buying a **Ford Fusion** are likely negatively correlated patterns
- ❑ How to define negative patterns?

Defining Negative Correlated Patterns

- A support-based definition
 - If itemsets A and B are both frequent but rarely occur together, i.e.,
 - $\text{sup}(A \cup B) \ll \text{sup}(A) \times \text{sup}(B)$
 - Then A and B are negatively correlated
- Is this a good definition for large transaction datasets?
- Ex.: Suppose a store sold two needle packages A and B 100 times each, but only one transaction contained both A and B
 - When there are in total 200 transactions, we have
 - $s(A \cup B) = 0.005, s(A) \times s(B) = 0.25, s(A \cup B) \ll s(A) \times s(B)$
 - But when there are 10^5 transactions, we have
 - $s(A \cup B) = 1/10^5, s(A) \times s(B) = 1/10^3 \times 1/10^3, s(A \cup B) > s(A) \times s(B)$
- What is the problem here?
 - Null transactions: The support-based definition is not null-invariant!

Does this remind you the definition of *lift*?

Defining Negative Correlation: Need Null-Invariance in Definition

- A good definition on negative correlation should be null-invariant!
 - Intuition: Whether two itemsets A and B are negatively correlated should not be influenced by the number of null-transactions
- A Kulczynski measure-based definition
 - If itemsets A and B are frequent but $(s(A \cup B)/s(A) + s(A \cup B)/s(B))/2 < \epsilon$, then A and B are negatively correlated
- For the same needle package problem:
 - No matter there are in total 200 or 10^5 transactions
 - If $\epsilon = 0.01$, we have
$$(s(A \cup B)/s(A) + s(A \cup B)/s(B))/2 = (0.01 + 0.01)/2 < \epsilon$$

ϵ is a negative pattern threshold

Pattern Mining: Advanced Methods

- ☐ Mining Various Kinds of Patterns
- ☐ Mining Compressed or Approximate Patterns 
- ☐ Constraint-based Pattern Mining
- ☐ Mining Sequential Patterns
- ☐ Mining Subgraph Patterns
- ☐ Pattern Mining: Application Examples
- ☐ Summary

Mining Compressed Patterns

Pat-ID	Item-Sets	Support
P1	{38,16,18,12}	205227
P2	{38,16,18,12,17}	205211
P3	{39,38,16,18,12,17}	101758
P4	{39,16,18,12,17}	161563
P5	{39,16,18,12}	161576

- ❑ Closed patterns
 - ❑ P1, P2, P3, P4, P5
 - ❑ Emphasizes too much on support
- ❑ Max-patterns
 - ❑ P3: information loss
- ❑ Desired output (a good balance):
 - ❑ P2, P3, P4

- ❑ Why mining compressed patterns? Too many scattered patterns but not so meaningful

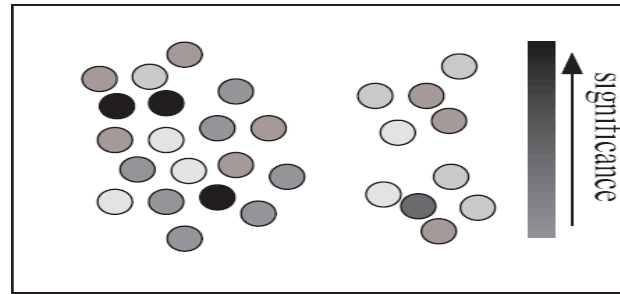
- ❑ Pattern distance measure

$$Dist(P_1, P_2) = 1 - \frac{|T(P_1) \cap T(P_2)|}{|T(P_1) \cup T(P_2)|}$$

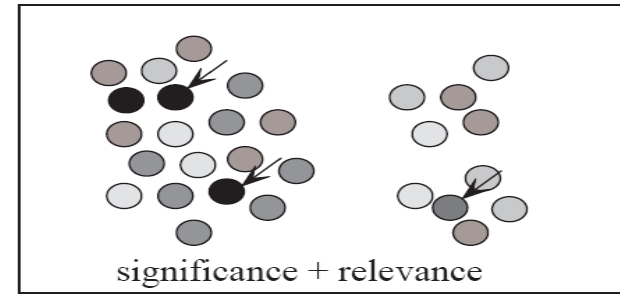
- ❑ δ -clustering: For each pattern P, find all patterns which can be expressed by P and whose distance to P is within δ (δ -cover)
- ❑ All patterns in the cluster can be represented by P

Redundancy-Aware Top-k Patterns

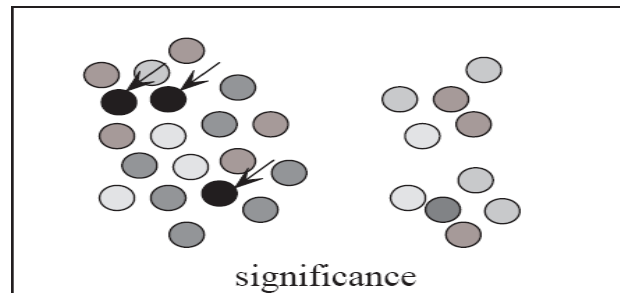
- Desired patterns: high significance & low redundancy



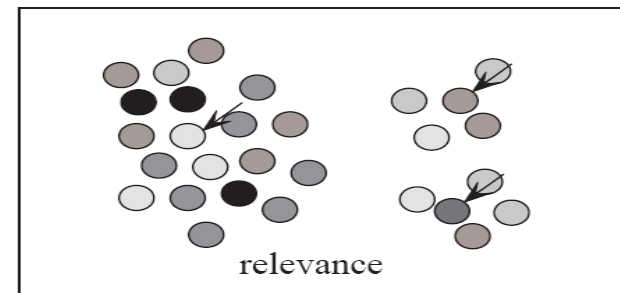
(a) a set of patterns



(b) redundancy-aware top- k



(c) traditional top- k



(d) summarization

- Method: Use MMS (Maximal Marginal Significance) for measuring the combined significance of a pattern set
- Xin et al., Extracting Redundancy-Aware Top-K Patterns, KDD'06

Pattern Mining: Advanced Methods

- ☐ Mining Various Kinds of Patterns
- ☐ Mining Compressed or Approximate Patterns
- ☐ Constraint-based Pattern Mining 
- ☐ Mining Sequential Patterns
- ☐ Mining Subgraph Patterns
- ☐ Pattern Mining: Application Examples
- ☐ Summary

Why Constraint-Based Mining?

- ❑ Pattern mining in practice: Often a user-guided, **interactive** process
 - ❑ User directs what to be mined using a **data mining query language** (or a graphical user interface), **specifying various kinds of constraints**

- ❑ What is constraint-based mining?
 - ❑ Mine together with user-provided constraints

- ❑ Why constraint-based mining?
 - ❑ User flexibility: User provides **constraints** on what to be mined
 - ❑ Optimization: System explores such constraints for mining efficiency
 - ❑ E.g., Push constraints deeply into the mining process

Various Kinds of User-Specified Constraints in Data Mining

- ❑ **Knowledge type constraint**—Specifying what kinds of knowledge to mine
 - ❑ E.g.: Classification, association, clustering, outlier finding, ...
- ❑ **Data constraint**—using SQL-like queries
 - ❑ E.g.: Find products sold together in NY stores this year
- ❑ **Dimension/level constraint**—similar to projection in relational database
 - ❑ E.g.: In relevance to region, price, brand, customer category
- ❑ **Interestingness constraint**—various kinds of thresholds
 - ❑ E.g.: Strong rules: $\text{min_sup} \geq 0.02$, $\text{min_conf} \geq 0.6$, $\text{min_correlation} \geq 0.7$
- ❑ **Rule (or pattern) constraint**  **The focus of this study**
 - ❑ E.g.: Small sales (price < \$10) triggers big sales (sum > \$200)

Pattern Space Pruning with Pattern Anti-Monotonicity

TID	Transaction
10	a, b, c, d, f, h
20	b, c, d, f, g, h
30	b, c, d, f, g
40	a, c, e, f, g

min_sup = 2

Item	Price	Profit
a	100	40
b	40	0
c	150	-20
d	35	-15
e	55	-30
f	45	-10
g	80	20
h	10	5

■ A constraint c is **anti-monotone**

- If an itemset S **violates** constraint c , so does any of its superset
- That is, mining on itemset S can be terminated

■ Ex. 1: $c_1: \text{sum}(S.\text{price}) \leq v$ is **anti-monotone**

■ Ex. 2: $c_2: \text{range}(S.\text{profit}) \leq 15$ is **anti-monotone**

- Itemset ab violates c_2 ($\text{range}(ab) = 40$)

- So does every superset of ab

■ Ex. 3. $c_3: \text{sum}(S.\text{Price}) \geq v$ is **not anti-monotone**

■ Ex. 4. Is $c_4: \text{support}(S) \geq \sigma$ anti-monotone?

- Yes! Apriori pruning is essentially pruning with an anti-monotonic constraint!

Note: item.price > 0
Profit can be negative

Pattern Monotonicity and Its Roles

TID	Transaction
10	a, b, c, d, f, h
20	b, c, d, f, g, h
30	b, c, d, f, g
40	a, c, e, f, g

min_sup = 2

Item	Price	Profit
a	100	40
b	40	0
c	150	-20
d	35	-15
e	55	-30
f	45	-10
g	80	20
h	10	5

- A constraint c is *monotone*: If an itemset S **satisfies** the constraint c , so does any of its superset
 - That is, we do not need to check c in subsequent mining
 - Not as beneficial as anti-monotone
- E.g. 1: $c_1: \text{sum}(S.\text{Price}) \geq v$ is **monotone**
- E.g. 2: $c_2: \text{min}(S.\text{Price}) \leq v$ is **monotone**
- E.g. 3: $c_3: \text{range}(S.\text{profit}) \geq 15$ is **monotone**
 - Itemset ab satisfies c_3
 - So does every superset of ab

Apriori for Pattern Anti-Monotone Constraint

Item	Price
1	1
2	2
3	3
4	4
5	5

Database D

TID	Items
10	1 3 4
20	2 3 5
30	1 2 3 5
40	2 5

Scan D

itemset	sup.
{1}	2
{2}	3
{3}	3
{4}	1
{5}	3

F_1

itemset	sup.
{1}	2
{2}	3
{3}	3
{5}	3

Can be
chopped
early

F_2

itemset	sup
{1 3}	2
{2 3}	2
{2 5}	3
{3 5}	2

C_2

itemset	sup
{1 2}	1
{1 3}	2
{1 5}	1
{2 3}	2
{2 5}	3
{3 5}	2

C_2

itemset	sup
{1 2}	1
{1 3}	2
{1 5}	1
{2 3}	2
{2 5}	3
{3 5}	2

C_3

itemset	sup
{2 3 5}	2

Scan D

itemset	sup
{2 3 5}	2

Min_sup=2
Constraint:
Sum{S.price} < 5

Convertible Constraints: Ordering Data in Transactions

TID	Transaction
10	a, b, c, d, f, h
20	a, b, c, d, f, g, h
30	b, c, d, f, g
40	a, c, e, f, g

min_sup = 2

Item	Price	Profit
a	100	40
b	40	0
c	150	-20
d	35	-15
e	55	-30
f	45	-5
g	80	30
h	10	5

- Convert tough constraints into (anti-)monotone by proper ordering of items in transactions
- Examine c_1 : $\text{avg}(S.\text{profit}) > 20$
 - Order items in (profit) value-descending order
 - $\langle a, g, h, b, f, d, c, e \rangle$
 - An itemset ab violates c_1 ($\text{avg}(ab) = 20$)
 - So does ab^* (i.e., ab -projected DB)
 - C_1 : **anti-monotone if patterns grow in the right order!**

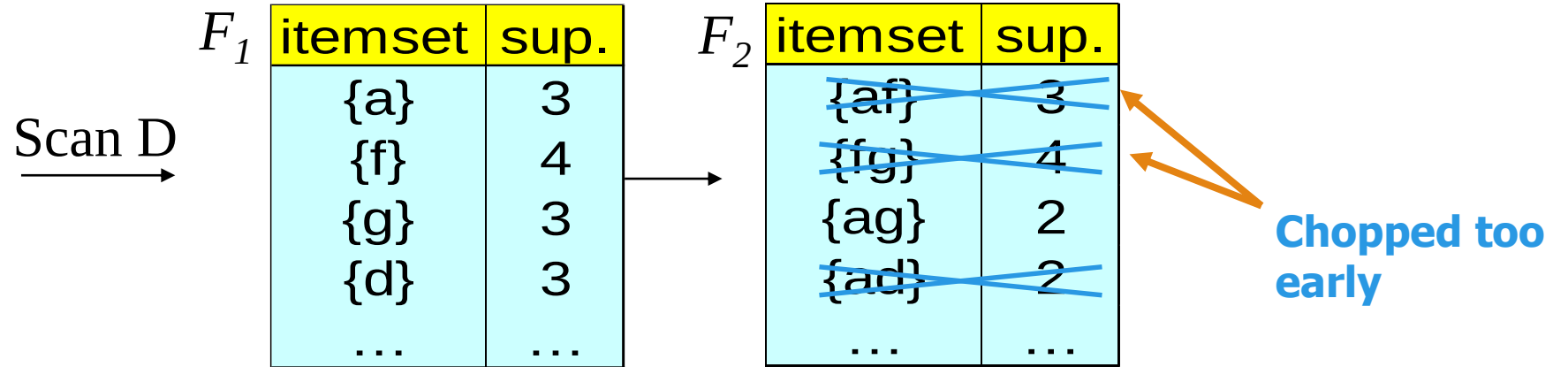
Can item-reordering work for Apriori?

TID	Transaction
10	a, b, c, d, f, h
20	a, b, c, d, f, g, h
30	b, c, d, f, g
40	a, c, e, f, g

min_sup = 2

Item	Price	Profit
a	100	40
b	40	0
c	150	-20
d	35	-15
e	55	-30
f	45	-5
g	80	30
h	10	5

constraint: $\text{avg}(\text{S.profit}) > 20$



- $\text{avg}(gf) = 12.5 < 20$, $\text{avg}(af) = 17.5 < 20$, $\text{avg}(ag) = 35 > 20$
- But $\text{avg}(agf) = 21.7 > 20$
- Apriori will not generate “agf” as a candidate

Data Space Pruning with Data Anti-Monotonicity

TID	Transaction
10	a, b, c, d, f, h
20	b, c, d, f, g, h
30	b, c, d, f, g
40	a, c, e, f, g

min_sup = 2

Item	Price	Profit
a	100	40
b	40	0
c	150	-20
d	35	-15
e	55	-30
f	45	-10
g	80	20
h	10	5

- A constraint c is **data anti-monotone**: In the mining process, if a data entry t cannot satisfy a pattern p under c , t cannot satisfy p 's superset either
 - Data space pruning: Data entry t can be pruned
- Ex. 1: $c_1: \text{sum}(S.\text{Profit}) \geq v$ is **data anti-monotone**
 - Let constraint c_1 be: $\text{sum}(S.\text{Profit}) \geq 25$
 - $T_{30}: \{b, c, d, f, g\}$ can be removed since none of their combinations can make an S whose sum of the profit is ≥ 25
- Ex. 2: $c_2: \text{min}(S.\text{Price}) \leq v$ is **data anti-monotone**
 - Consider $v = 5$ but every item in a transaction, say T_{50} , has a price higher than 10
- Ex. 3: $c_3: \text{range}(S.\text{Profit}) > 25$ is **data anti-monotone**

Data Space Pruning Should Be Explored Recursively

TID	Transaction	Item	Profit
10	a, b, c, d, f, h	a	40
20	b, c, d, f, g, h	b	0
30	b, c, d, f, g	c	-20
40	a, c, e, f, g	d	-15
min_sup = 2		e	-30
		f	-10
		g	20
		h	5

- ❑ Example. $c_3: \text{range}(S.\text{Profit}) > 25$
 - ❑ We check b's projected database
 - ❑ But item "a" is infrequent (sup = 1)



b's-proj. DB

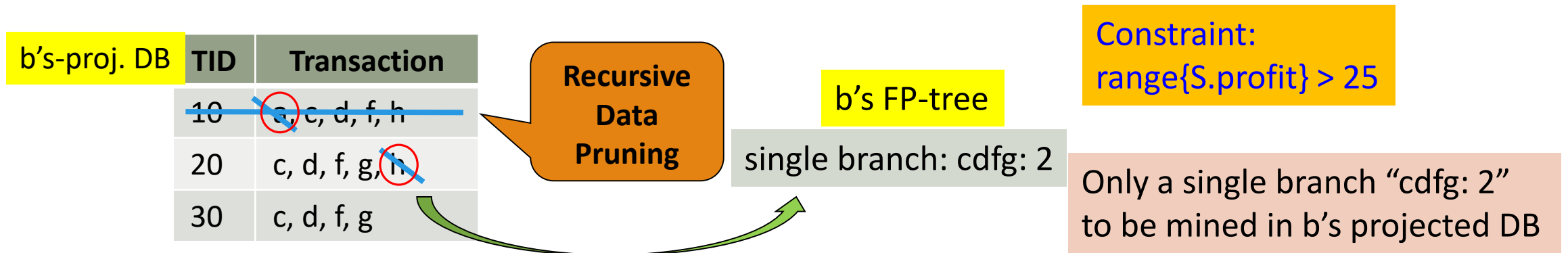
TID	Transaction
10	a, c, d, f, h
20	c, d, f, g, h
30	c, d, f, g

- ❑ After removing "a (40)" from T_{10}
 - ❑ T_{10} cannot satisfy c_3 any more
 - ❑ Since "b (0)" and "c (-20), d (-15), f (-10), h (5)"
 - ❑ By removing T_{10} , we can also prune "h" in T_{20}

b's-proj. DB

TID	Transaction
10	a, c, d, f, h
20	c, d, f, g, h
30	c, d, f, g

Data Space Pruning Explored Recursively (Continued)



- Note: c_3 prunes T_{10} effectively only after "a" is pruned (by min-sup) in b's projected DB

Succinctness: Pruning Both Data and Pattern Spaces

- Succinctness: If the constraint c can be enforced by directly manipulating the data
- E.g. 1: To find those patterns containing item i
 - Mine only i -projected DB (data space pruning)
- E.g. 2: To find those patterns without item i
 - Remove i from DB and then mine (pattern space pruning)
- E.g. 3: $c_3: \min(S.Price) \leq v$ is succinct
 - Start with only items whose price $\leq v$ and remove transactions with high-price items only (pattern + data space pruning)
- E.g. 4: $c_4: \sum(S.Price) \geq v$ is not succinct
 - It cannot be determined beforehand since sum of the price of itemset S keeps increasing

Constrained FP-Growth: Push a Succinct Constraint Deep

Item	Price
1	1
2	2
3	3
4	4
5	5

TID	Items
10	1 3 4
20	2 3 5
30	1 2 3 5
40	2 5

Remove
infrequent
length 1

TID	Items
10	1 3
20	2 3 5
30	1 2 3 5
40	2 5

Min_sup=2

Constraint:

$\min\{S.\text{price}\} \leq 2$

No Need to project on 3 or 5

1-Projected DB

TID	Items
10	3 4
30	2 3 5

2-Projected DB

TID	Items
20	3 5
30	1 3 5
40	5

Different Kinds of Constraints Lead to Different Pruning Strategies

- In summary, constraints can be categorized as **pattern space pruning** constraints vs. **data space pruning** constraints

Pattern space pruning constraints	Data space pruning constraints
■ Anti-monotonic: If constraint c is violated, its further mining can be terminated ■ Monotonic: If c is satisfied, no need to check c again ■ Convertible: c can be converted to monotonic or anti-monotonic if items can be properly ordered in processing ■ Succinct: If the constraint c can be enforced by directly manipulating the data	■ Data succinct: Data space can be pruned at the initial pattern mining process ■ Data anti-monotonic: If a transaction t does not satisfy c, then t can be pruned to reduce data processing effort

How to Handle Multiple Constraints?

- It is beneficial to use multiple constraints in pattern mining
- But different constraints may require potentially conflicting item-ordering
 - If there exists conflict ordering between c_1 and c_2
 - Try to sort data and enforce *one constraint* first (which one?)
 - Then enforce the other constraint when mining the projected databases
- E.g. c_1 : $\text{avg}(S.\text{profit}) > 20$, and c_2 : $\text{avg}(S.\text{price}) < 50$
 - Assume c_1 has more pruning power
 - Sort in profit descending order and use c_1 first
 - For each project DB, sort trans. in price ascending order and use c_2 at mining

Pattern Mining: Advanced Methods

- ☐ Mining Various Kinds of Patterns
- ☐ Mining Compressed or Approximate Patterns
- ☐ Constraint-based Pattern Mining
- ☐ Mining Sequential Patterns 
- ☐ Mining Subgraph Patterns
- ☐ Pattern Mining: Application Examples
- ☐ Summary

Sequential Pattern Mining

- ❑ Sequential Pattern and Sequential Pattern Mining
- ❑ GSP: Apriori-Based Sequential Pattern Mining
- ❑ SPADE: Sequential Pattern Mining in Vertical Data Format
- ❑ PrefixSpan: Sequential Pattern Mining by Pattern-Growth
- ❑ CloSpan: Mining Closed Sequential Patterns
- ❑ Constraint-Based Sequential-Pattern Mining

Sequential Pattern Mining

- ❑ What kind of patterns are sequential?
- ❑ Sequential: The order really matters
 - ❑ You cannot swap two items in a sequence and have the same sequence
- ❑ Example: The English language is sequential: Subject → Verb → Object

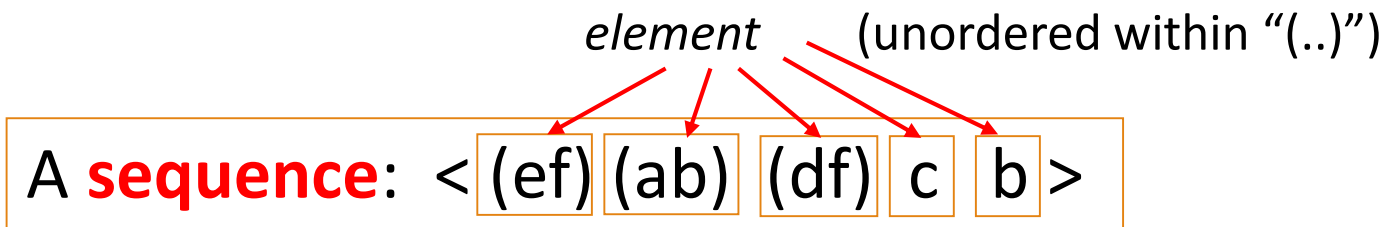
- ❑ Other points:
 - ❑ For Sequential Pattern Mining, the time which the items occur is **not** considered
 - ❑ Time Series Analysis does take into account the time in which an item occurred

Sequential Pattern Examples

- ❑ Application of Sequential pattern Mining
 - ❑ **Customer shopping:** Purchase a laptop first, then a digital camera, and then a smartphone
 - ❑ **Medical treatments:** Go to see a doctor, get drugs, doctor monitors progress, doctor reacts accordingly → more/less drugs
 - ❑ **Natural disasters:** Before the disaster, during the disaster, after the disaster
 - ❑ **Scientific Experiments:** Step 1, Step 2, Step 3
 - ❑ **Stocks Markets:** A set of stocks go up and down together
 - ❑ **Biological sequences, DNA /Protein:** If you change the order of a protein, it likely results in a different gene

Sequential Pattern and Sequential Pattern Mining

- Sequential pattern mining: Given a set of sequences, find the **complete set of frequent subsequences** (i.e., satisfying the min_sup threshold)



- An element may contain a set of items (also called events)

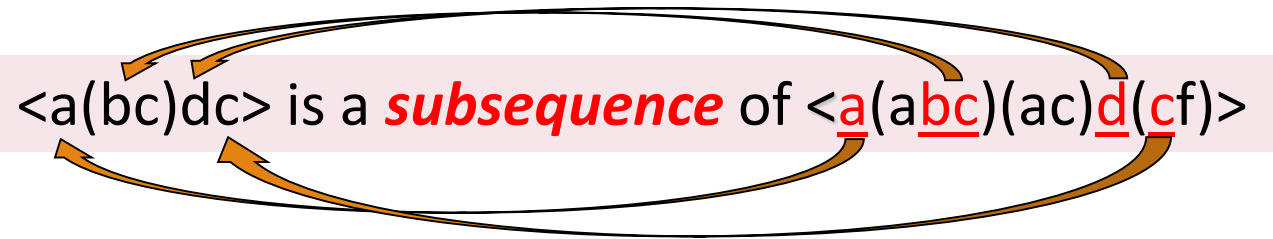
* Items within an element are **unordered** and we list them alphabetically

A **sequence database**

SID	Sequence
10	<a(<u>ab</u> c)(a <u>c</u>)d(cf)>
20	<(ad)c(bc)(ae)>
30	<(ef)(<u>ab</u>)(df) <u>c</u> b>
40	<eg(af)cbc>

Sequential Pattern and Sequential Pattern Mining

- **Sequential pattern mining:** Given a set of sequences, find the **complete set of frequent subsequences** (i.e., satisfying the min_sup threshold)



- Given support threshold $min_sup = 2$, $<(ab)c>$ is a sequential pattern

A **sequence database**

SID	Sequence
10	$<a(\underline{ab}c)(a\underline{c})d(cf)>$
20	$<(ad)c(bc)(ae)>$
30	$<(ef)(\underline{ab})(df)\underline{c}b>$
40	$<eg(af)cbc>$

Sequential Pattern Mining Algorithms

- ❑ Algorithm requirement: Efficient, scalable, finding complete set, incorporating various kinds of user-specific constraints
- ❑ The Apriori property still holds: If a subsequence s_1 is infrequent, none of s_1 's super-sequences can be frequent
- ❑ Representative algorithms
 - ❑ **GSP** (Generalized Sequential Patterns): Srikant & Agrawal @ EDBT'96)
 - ❑ Vertical format-based mining: **SPADE** (Zaki@Machine Learning'00)
 - ❑ Pattern-growth methods: **PrefixSpan** (Pei, et al. @TKDE'04)
- ❑ Mining closed sequential patterns: **CloSpan** (Yan, et al. @SDM'03)
- ❑ Constraint-based sequential pattern mining

GSP: Apriori-Based Sequential Pattern Mining

- Initial candidates: All 8-singleton sequences
 - $\langle a \rangle, \langle b \rangle, \langle c \rangle, \langle d \rangle, \langle e \rangle, \langle f \rangle, \langle g \rangle, \langle h \rangle$
- Scan DB once, count support for each candidate

SID	Sequence
10	$\langle (bd)cb(ac) \rangle$
20	$\langle (bf)(ce)b(fg) \rangle$
30	$\langle (ah)(bf)abf \rangle$
40	$\langle (be)(ce)d \rangle$
50	$\langle a(bd)bcb(ade) \rangle$

$min_sup = 2$

Cand.	sup
$\langle a \rangle$	3
$\langle b \rangle$	5
$\langle c \rangle$	4
$\langle d \rangle$	3
$\langle e \rangle$	3
$\langle f \rangle$	2
$\langle g \rangle$	1
$\langle h \rangle$	1



GSP (Generalized Sequential Patterns): Srikant & Agrawal @ EDBT'96)

GSP: Apriori-Based Sequential Pattern Mining

- Example: Generate length-2 candidate sequences

singleton * singleton – Total: (6 * 6)

	<a>		<c>	<d>	<e>	<f>
<a>	<aa>	<ab>	<ac>	<ad>	<ae>	<af>
	<ba>	<bb>	<bc>	<bd>	<be>	<bf>
<c>	<ca>	<cb>	<cc>	<cd>	<ce>	<cf>
<d>	<da>	<db>	<dc>	<dd>	<de>	<df>
<e>	<ea>	<eb>	<ec>	<ed>	<ee>	<ef>
<f>	<fa>	<fb>	<fc>	<fd>	<fe>	<ff>

$min_sup = 2$

Cand.	sup
<a>	3
	5
<c>	4
<d>	3
<e>	3
<f>	2
<g>	1
<h>	1



Sets (unordered) – Total: (6*5) / 2

	<a>		<c>	<d>	<e>	<f>
<a>		<(ab)>	<(ac)>	<(ad)>	<(ae)>	<(af)>
			<(bc)>	<(bd)>	<(be)>	<(bf)>
<c>				<(cd)>	<(ce)>	<(cf)>
<d>					<(de)>	<(df)>
<e>						<(ef)>
<f>						

Apriori Pruning

- w/o pruning
(includes g and h):

$$8*8 + 8*7/2 = 92$$

length-2 candidates

- w/ pruning:

$$6*6 + 6*5/2 = 51$$

length-2 candidates

GSP Mining and Pruning

			<u>length</u>
5 th scan: 1 cand. 1 length-5 seq. pat.	<(bd)cba>	☐ Remove Candidates not in DB or Candidates < min_sup	5
4 th scan: 8 cand. 7 length-4 seq. pat.	<abba> <(bd)bc> ...		4
3 rd scan: 46 cand. 20 length-3 seq. pat. 20 cand. not in DB at all	<abb> <aab> <aba> <baa> <bab> ...		3
2 nd scan: 51 cand. 19 length-2 seq. pat. 10 cand. not in DB at all	<aa> <ab> ... <af> <ba> <bb> ... <ff> <(ab)> ... <(ef)>		2
1 st scan: 8 cand. 6 length-1 seq. pat.	<a> <c> <d> <e> <f> <g> <h>		1

$$6*6 + 6*5/2 = 51$$

The GPS algorithm

- ☐ Repeat, starting at k = 1 until k <= length
 - ☐ Scan DB to find “length-k” frequent sequences
 - ☐ Generate “length-(k+1)” candidate sequences from “length-k” frequent sequences using **Apriori**
 - ☐ set k = k+1
- ☐ Until no frequent sequence or no candidate can be found

min_sup = 2

SID	Sequence
10	<(bd)cb(ac)>
20	<(bf)(ce)b(fg)>
30	<(ah)(bf)abf>
40	<(be)(ce)d>
50	<a(bd)bcb(ade)>

Sequential Pattern Mining in Vertical Data Format: The SPADE Algorithm

- A sequence database is mapped to: <SID, EID>
- Grow the subsequences (patterns) one item at a time by Apriori candidate generation

SID	Sequence
1	<a(<u>abc</u>)(a <u>c</u>)d(cf)>
2	<(ad)c(bc)(ae)>
3	<(ef)(<u>ab</u>)(df) <u>cb</u> >
4	<eg(af)cbc>

$min_sup = 2$

Ref: SPADE (Sequential Pattern Discovery using Equivalent Class) [M. Zaki 2001]

SID	EID	Items
1	1	a
1	2	abc
1	3	ac
1	4	d
1	5	cf
2	1	ad
2	2	c
2	3	bc
2	4	ae
3	1	ef
3	2	ab
3	3	df
3	4	c
3	5	b
4	1	e
4	2	g
4	3	af
4	4	c
4	5	b
4	6	c

a		b		...
SID	EID	SID	EID	...
1	1	1	2	
1	2	2	3	
1	3	3	2	
2	1	3	5	
2	4	4	5	
3	2			
4	3			

EID (b) < EID (a):
Corresponds to:
<a(abc)(ac)d(cf)>

ab			ba			...
SID	EID (a)	EID(b)	SID	EID (b)	EID(a)	...
1	1	2	1	2	3	
2	1	3	2	3	4	
3	2	5				
4	3	5				

aba				...
SID	EID (a)	EID(b)	EID(a)	...
1	1	2	3	
2	1	3	4	

PrefixSpan: A Pattern-Growth Approach

SID	Sequence	<i>min_sup</i> = 2	
		Prefix	Suffix (Projection)
10	<a(<u>abc</u>)(ac)d(cf)>	<a>	<(abc)(ac)d(cf)>
20	<(ad)c(bc)(ae)>	<aa>	<(_bc)(ac)d(cf)>
30	<(ef)(<u>ab</u>)(df) <u>cb</u> >	<ab>	<(_c)(ac)d(cf)>
40	<eg(af)cbc>		

PrefixSpan Mining: Prefix Projections

Step 1: Find length-1 sequential patterns

<a>, , <c>, <d>, <e>, <f>

Step 2: Divide search space and mine each projected DB

<a>-projected DB,

-projected DB,

...

<f>-projected DB, ...

Prefix and suffix

Given <a(abc)(ac)d(cf)>

Prefixes: <a>, <aa>, <a(ab)>, <a(abc)>, ...

Suffix: Prefixes-based projection

"_" is placeholder for prefix

PrefixSpan (Prefix-projected Sequential pattern mining)
Pei, et al. @TKDE'04

PrefixSpan: Mining Prefix-Projected DBs

SID	Sequence
10	<a(<u>abc</u>)(<u>ac</u>)d(cf)>
20	<(ad)c(bc)(ae)>
30	<(ef)(<u>ab</u>)(df) <u>cb</u> >
40	<eg(af)cbc>

$min_sup = 2$

Length-1 sequential patterns
<a>, , <c>, <d>, <e>, <f>

prefix <a>

<a>-projected DB

<(abc)(ac)d(cf)>

<(_d)c(bc)(ae)>

<(_b)(df)cb>

<(_f)cbc>

Length-2 sequential patterns
<aa>, <ab>, <(ab)>,
<ac>, <ad>, <af>

prefix

-projected DB

<(_c)(ac)d(cf)>

<(_c)(ae)>

<(df)cb>

<c>

prefix <c>, ..., <f>

...

... ..

prefix <aa>

<aa>-projected DB

prefix <af>

<af>-projected DB

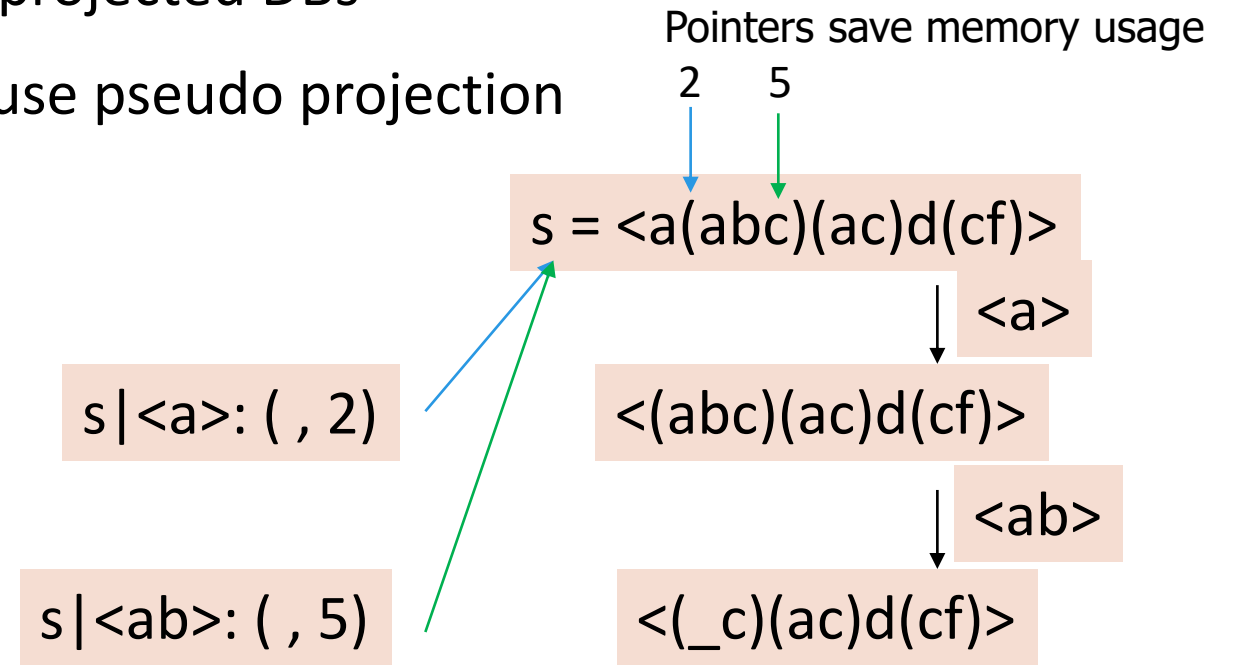
..

Major strength of PrefixSpan:

- No candidate subseqs. to be generated
- Projected DBs keep shrinking

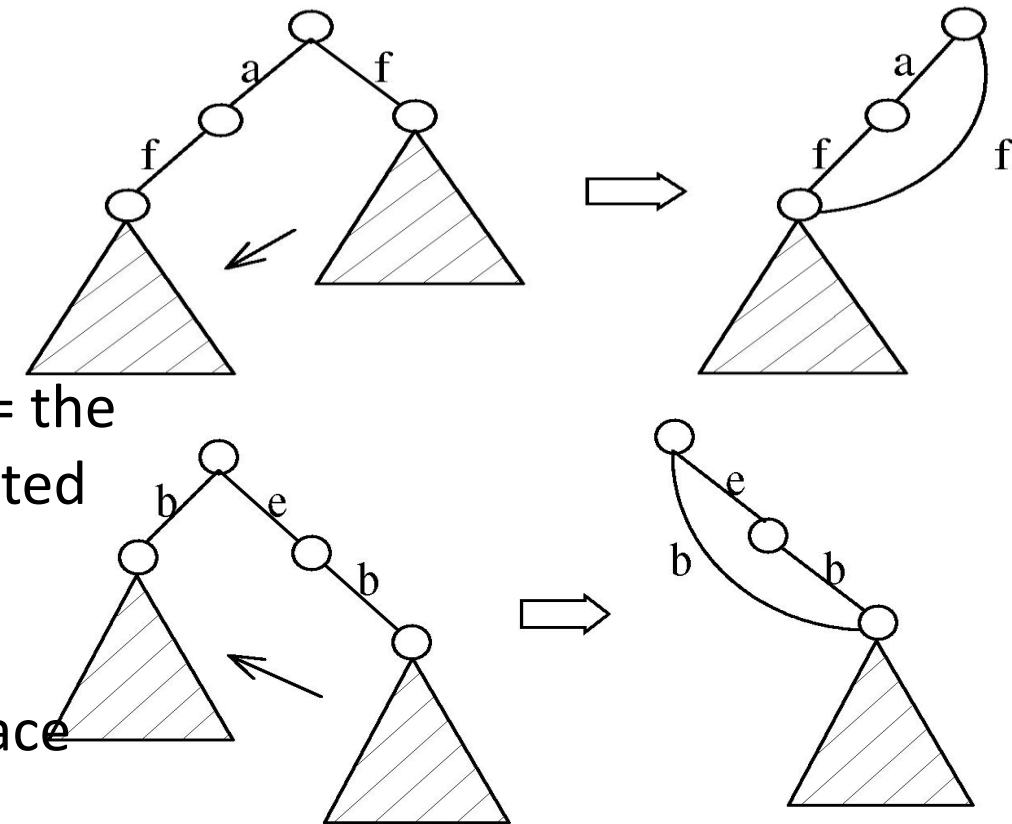
Implementation Consideration: Pseudo-Projection vs. Physical Projection

- ❑ Major cost of PrefixSpan: Constructing projected DBs
 - ❑ Suffixes largely repeating in recursive projected DBs
- ❑ When DB can be held in main memory, use pseudo projection
 - ❑ No physically copying suffixes
 - ❑ **Pointer to the sequence**
 - ❑ **Offset of the suffix**
- ❑ But if it does not fit in memory
 - ❑ Physical projection
- ❑ Suggested approach:
 - ❑ Integration of physical and pseudo-projection
 - ❑ Swapping to pseudo-projection when the data fits in memory



CloSpan: Mining Closed Sequential Patterns

- ❑ A **closed sequential pattern** s : There exists no superpattern s' such that $s' \supset s$, and s' and s have the same support
- ❑ Which ones are closed? $\langle abc \rangle: 20$, $\langle abcd \rangle: 20$, $\langle abcde \rangle: 15$
- ❑ Why directly mine closed sequential patterns?
 - ❑ Reduce # of (redundant) patterns
 - ❑ Attain the same expressive power
- ❑ Property P: Given two sequences s and s' , if s is a subsequence of s' , then the projected database of $s =$ the projected database of s' iff the size of the two projected databases are the same.
- ❑ Explore **Backward Subpattern** and **Backward Superpattern** pruning to prune redundant search space
- ❑ Greatly enhances efficiency (Yan, et al., SDM'03)

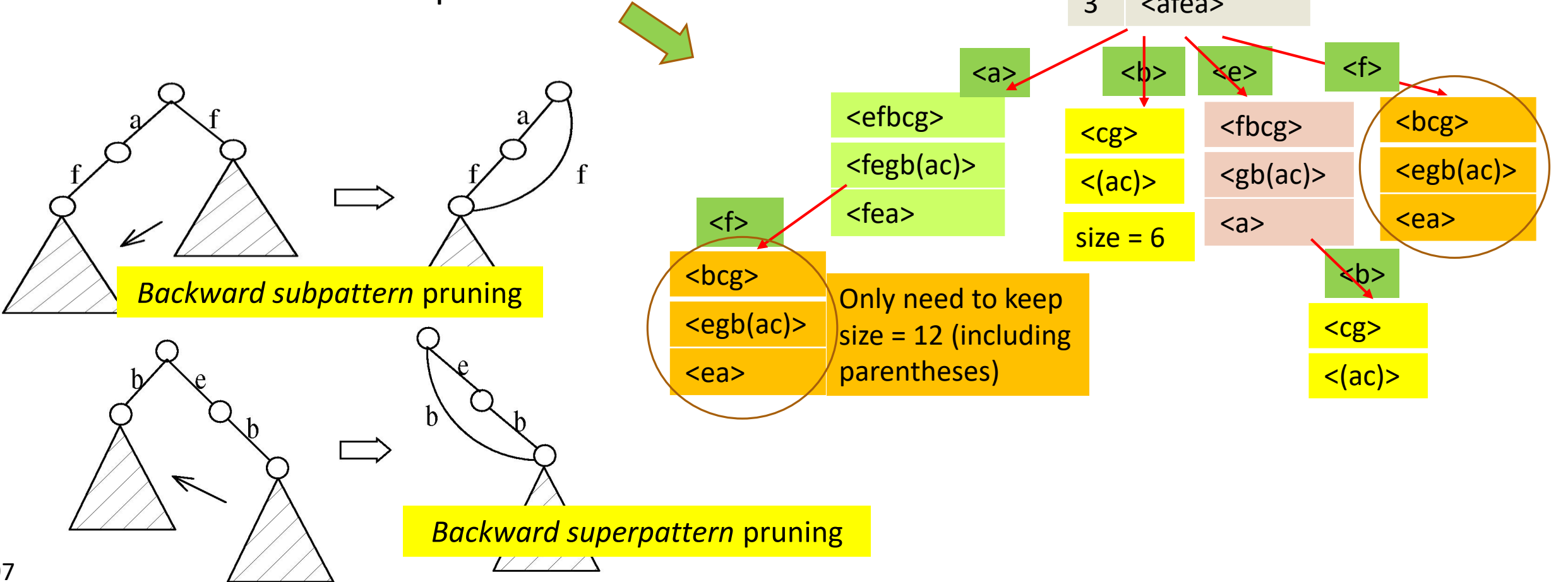


CloSpan: When Two Projected DBs Have the Same Size

- ❑ Exploring Property P for closed pattern mining
- ❑ When two projected sequence DBs have the same size?
 - ❑ Here is one example:

ID	Sequence
1	<ae fb cg>
2	<afeg b (ac)>
3	<af ea >

min_sup = 2



Constraint-Based Sequential-Pattern Mining

- ❑ Share many similarities with constraint-based itemset mining
- ❑ **Anti-monotonic:** If S violates c , the super-sequences of S also violate c
 - ❑ $\text{sum}(S.\text{price}) < 150; \min(S.\text{value}) > 10$
- ❑ **Monotonic:** If S satisfies c , the super-sequences of S also do so
 - ❑ $\text{element_count}(S) > 5; S \supseteq \{\text{PC}, \text{digital_camera}\}$
- ❑ **Data anti-monotonic:** If a sequence s_1 with respect to S violates c_3 , s_1 can be removed
 - ❑ $c_3: \text{sum}(S.\text{price}) \geq v$
- ❑ **Succinct:** Enforce constraint c by explicitly manipulating data
 - ❑ $S \supseteq \{\text{i-phone}, \text{MacAir}\}$
- ❑ **Convertible:** Projection based on the sorted value not sequence order
 - ❑ $\text{value_avg}(S) < 25; \text{profit_sum}(S) > 160$
 - ❑ $\text{max}(S)/\text{avg}(S) < 2; \text{median}(S) - \text{min}(S) > 5$

Timing-Based Constraints in Seq.-Pattern Mining

- ❑ **Order constraint:** Some items must happen before the other
 - ❑ {algebra, geometry} $\rightarrow$ {calculus} (where “ $\rightarrow$ ” indicates ordering)
 - ❑ Anti-monotonic: Constraint-violating sub-patterns pruned
- ❑ **Min-gap/max-gap constraint:** Confines two elements in a pattern
 - ❑ E.g., mingap = 1, maxgap = 4
 - ❑ Succinct: Enforced directly during pattern growth
- ❑ **Max-span constraint:** Maximum allowed time difference between the 1st and the last elements in the pattern
 - ❑ E.g., maxspan (S) = 60 (days)
 - ❑ Succinct: Enforced directly when the 1st element is determined
- ❑ **Window size constraint:** Events in an element do not have to occur at the same time: Enforce max allowed time difference
 - ❑ E.g., window-size = 2: Various ways to merge events into elements

Episodes and Episode Pattern Mining

- Episodes and regular expressions: Alternative to seq. patterns
 - Serial episodes: AB ← a total order relationship: first A then B
 - Parallel episodes: $A|B$ ← a partial order relationship: A and B can be in any order
 - Regular expressions: $(A|B)C^*(DE)$ ← (DE) means D, E happen in the same time window
- E.g. Given a large shopping sequence database, one may like to find
 - Suppose the pattern order follows the template $(A|B)C^*(D E)$, and
 - Sum of the prices of A, B, C^* , D, and E is greater than \$100, where C^* means C appears *-times
 - How to efficiently mine such episode patterns?

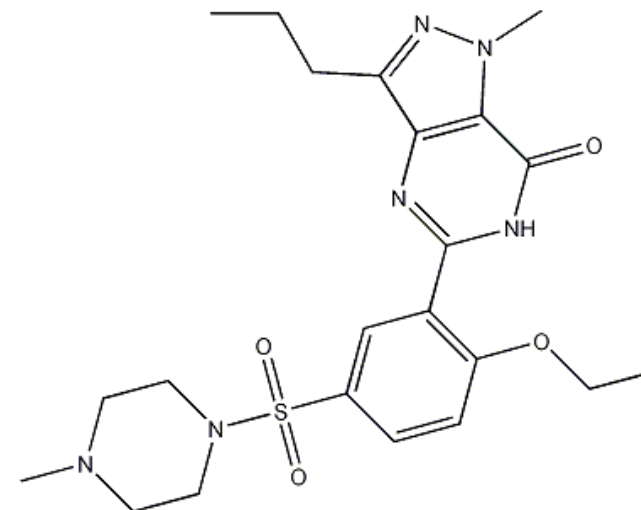
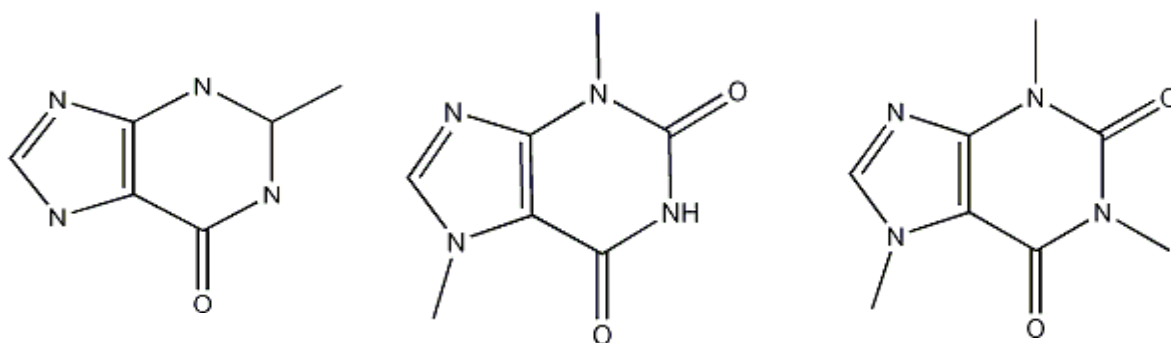
Pattern Mining: Advanced Methods

- ☐ Mining Various Kinds of Patterns
- ☐ Mining Compressed or Approximate Patterns
- ☐ Constraint-based Pattern Mining
- ☐ Mining Sequential Patterns
- ☐ Mining Subgraph Patterns 
- ☐ Pattern Mining: Application Examples
- ☐ Summary

What Is Graph Pattern Mining?

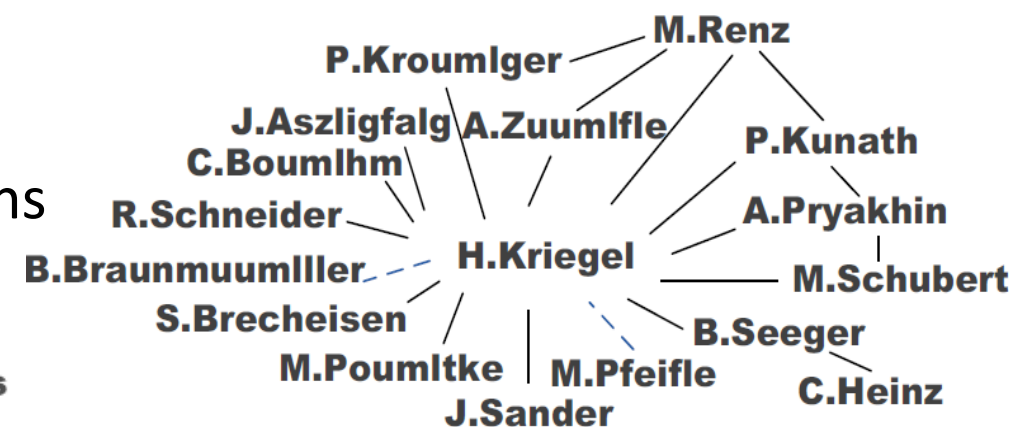
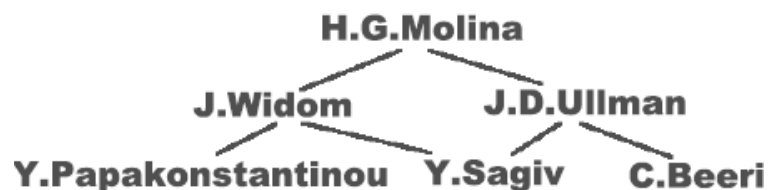
- Chem-informatics:

- Mining frequent chemical compound structures



- Social networks, web communities, tweets, ...

- Finding frequent research collaboration subgraphs



Frequent (Sub)Graph Patterns

- Given a labeled graph dataset $D = \{G_1, G_2, \dots, G_n\}$, the supporting graph set of a subgraph g is $D_g = \{G_i \mid g \subseteq G_i, G_i \in D\}$

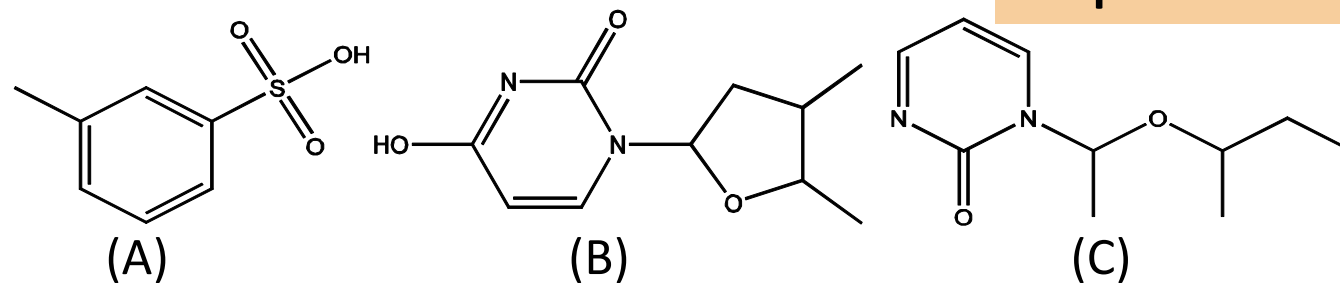
$\text{support}(g) = |D_g| / |D|$

- A (sub)graph g is **frequent** if $\text{support}(g) \geq \text{min_sup}$

- Ex.: Chemical structures

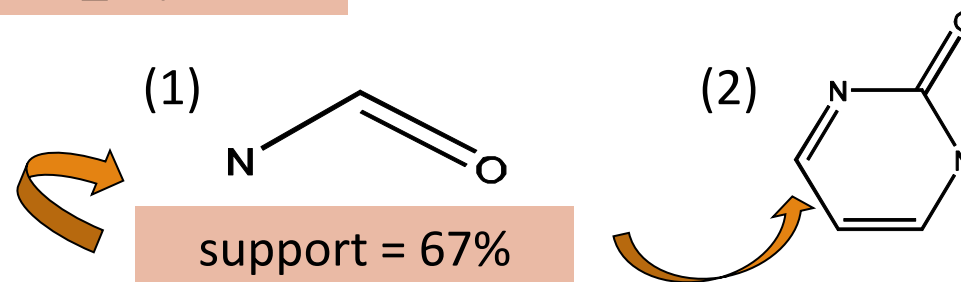
- Alternative:

- Mining frequent subgraph patterns from a single large graph or network



min_sup = 2

Frequent Graph Patterns



Applications of Graph Pattern Mining

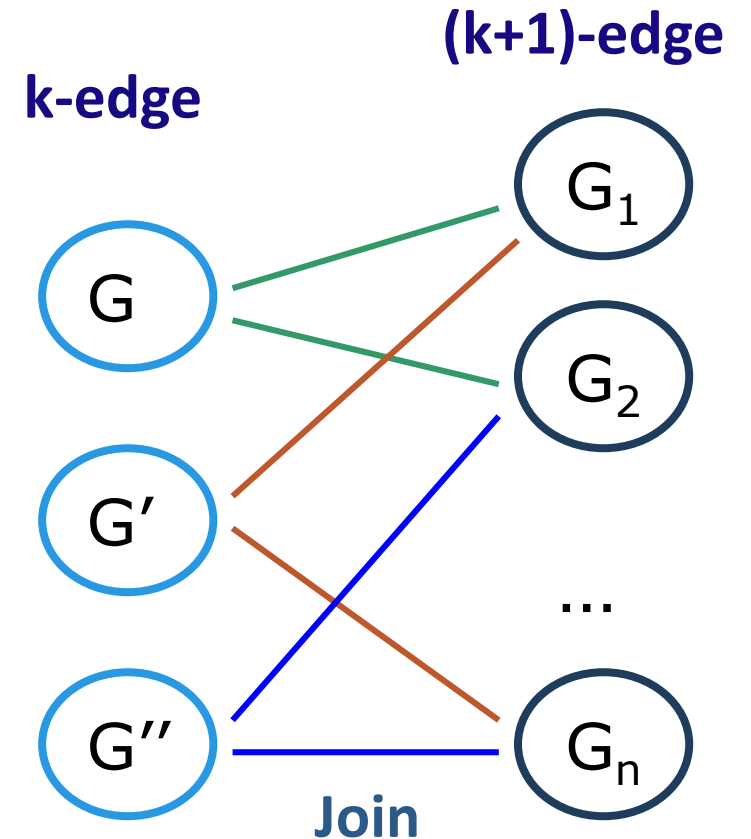
- ❑ Bioinformatics
 - ❑ Gene networks, protein interactions, metabolic pathways
- ❑ Chem-informatics: Mining chemical compound structures
- ❑ Social networks, web communities, tweets, ...
- ❑ Cell phone networks, computer networks, ...
- ❑ Web graphs, XML structures, Semantic Web, information networks
- ❑ Software engineering: Program execution flow analysis
- ❑ Building blocks for graph classification, clustering, compression, comparison, and correlation analysis
- ❑ Graph indexing and graph similarity search

Graph Pattern Mining Algorithms: Different Methodologies

- ❑ Generation of candidate subgraphs
 - ❑ Apriori vs. pattern growth (e.g., FSG vs. gSpan)
- ❑ Search order
 - ❑ Breadth vs. depth
- ❑ Elimination of duplicate subgraphs
 - ❑ Passive vs. active (e.g., gSpan [Yan & Han, 2002])
- ❑ Support calculation
 - ❑ Store embeddings (e.g., GASTON [Nijssen & Kok, 2004], FFSM [Huan, Wang, & Prins, 2003], MoFa [Borgelt & Berthold, ICDM'02])
- ❑ Order of pattern discovery
 - ❑ Path $\rightarrow$ tree $\rightarrow$ graph (e.g., GASTON [Nijssen & Kok, 2004])

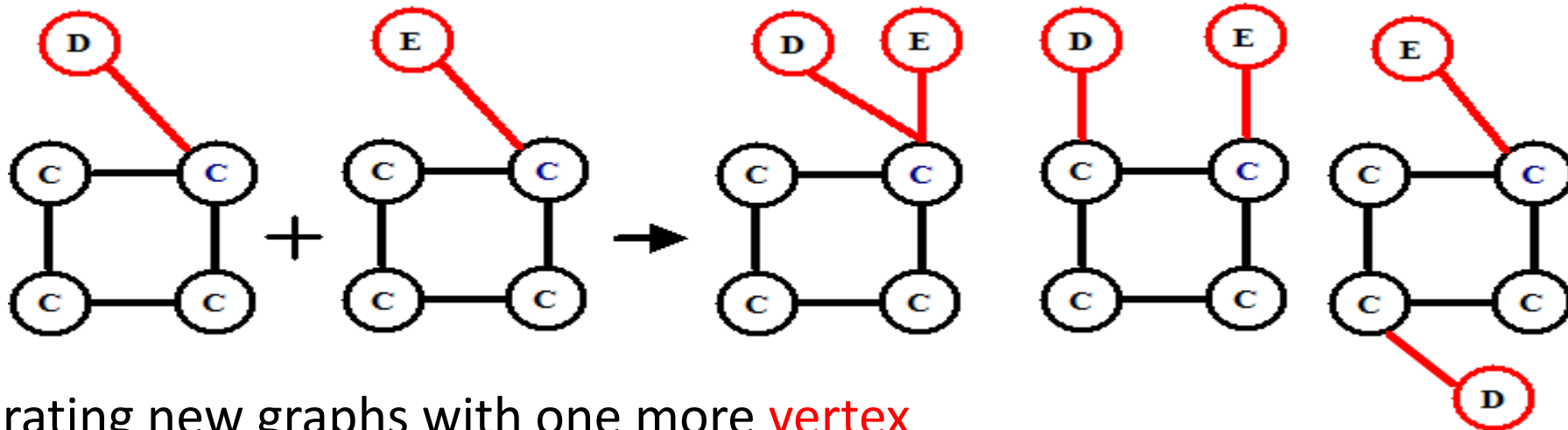
Apriori-Based Approach

- The Apriori property (anti-monotonicity): A size- k subgraph is **frequent** if and only if all of its **subgraphs are frequent**
- A candidate size- $(k+1)$ edge/vertex subgraph is generated if its corresponding two k -edge/vertex subgraphs are frequent
- Iterative mining process:
 - Candidate-generation $\rightarrow$ candidate pruning $\rightarrow$ support counting $\rightarrow$ candidate elimination



Candidate Generation: Vertex Growing vs. Edge Growing

- Methodology: Breadth-search, Apriori joining two size- k graphs
 - Many possibilities at generating size- $(k+1)$ candidate graphs



- Generating new graphs with one more **vertex**
 - AGM (Inokuchi, Washio, & Motoda, PKDD'00)
- Generating new graphs with one more **edge**
 - FSG (Kuramochi & Karypis, ICDM'01)
- Performance shows *via edge growing* is more efficient

Pattern-Growth Approach

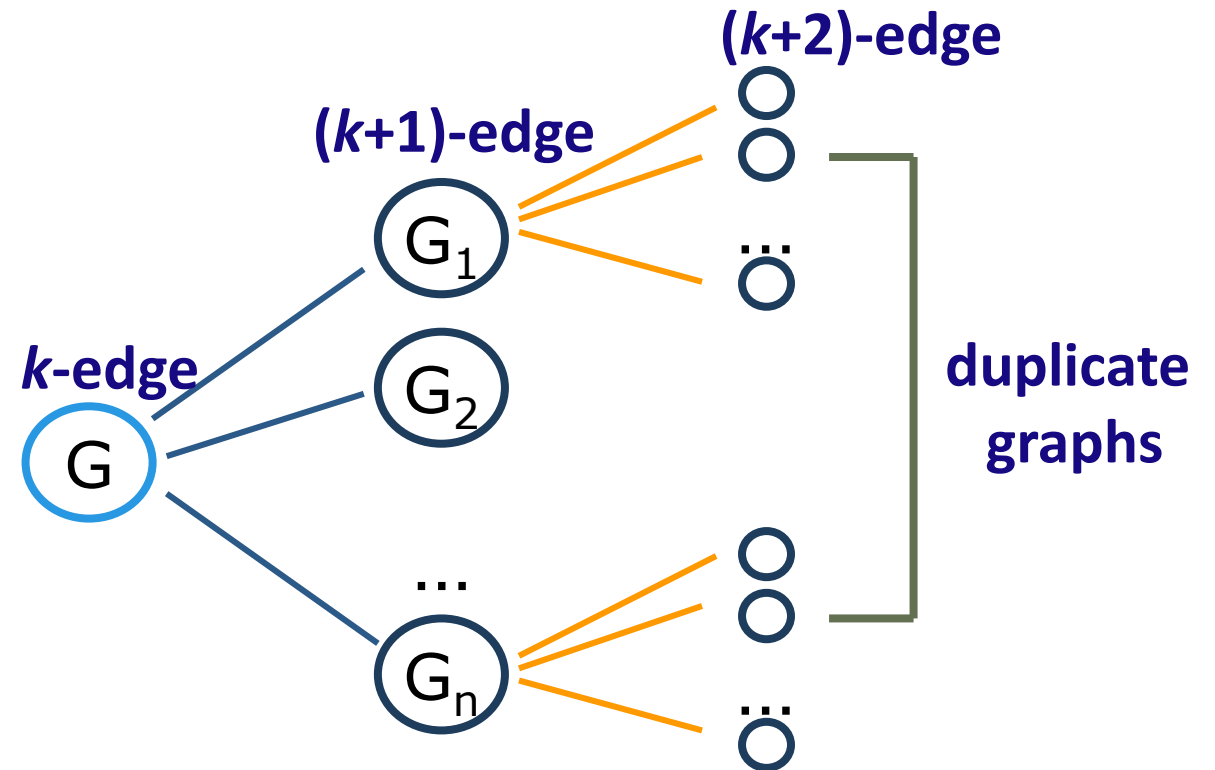
- Depth-first growth of subgraphs from k -edge to $(k+1)$ -edge, then $(k+2)$ -edge subgraphs

- Major challenge

- Generating many duplicate subgraphs

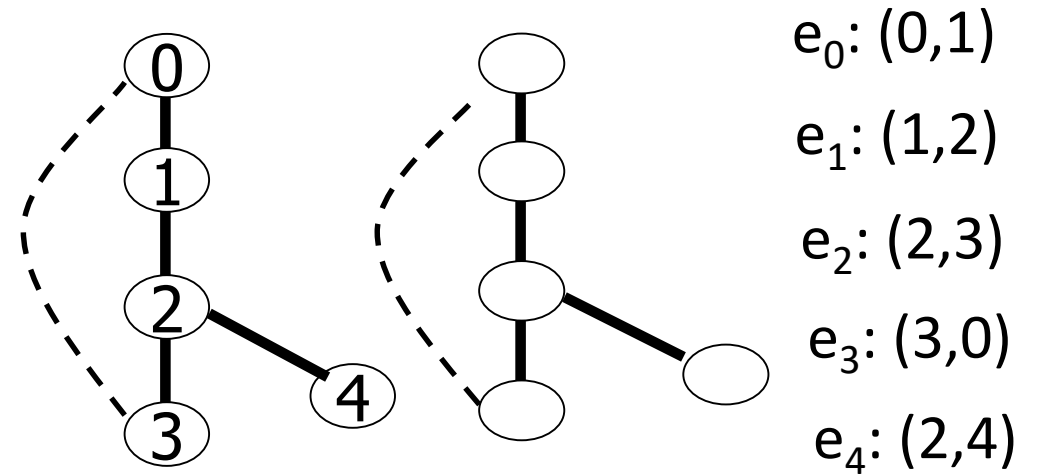
- Major idea to solve the problem

- Define an order to generate subgraphs
 - DFS spanning tree: Flatten a graph into a sequence using depth-first search
 - gSpan (Yan & Han, ICDM'02)



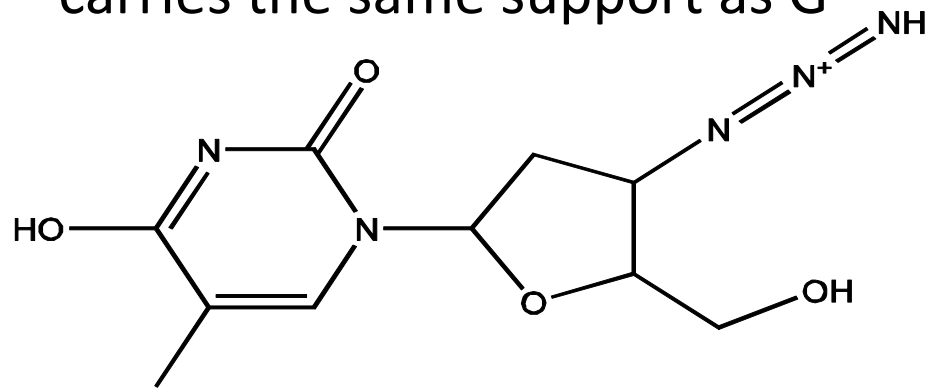
gSPAN: Graph Pattern Growth in Order

- ❑ **Right-most path extension** in subgraph pattern growth
 - ❑ Right-most path: The path from root to the right-most leaf (choose the vertex with the **smallest** index at each step)
 - ❑ Reduce generation of duplicate subgraphs
- ❑ **Completeness:** The enumeration of graphs using right-most path extension is complete
- ❑ DFS code: Flatten a graph into a sequence using depth-first search



Why Mine Closed Graph Patterns?

- 2^n subgraphs $\rightarrow$ *closed frequent subgraphs*
- A frequent graph G is *closed* if there exists no supergraph of G that carries the same support as G

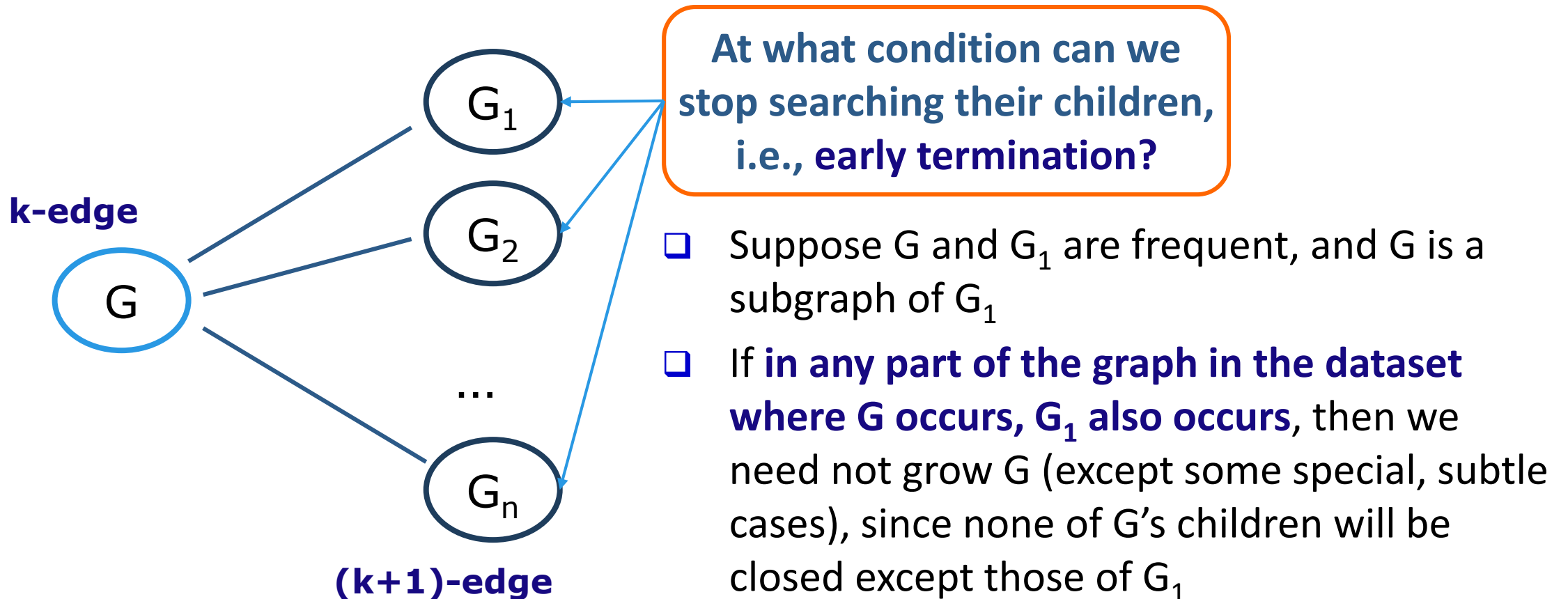


If this subgraph is *closed* in the graph dataset, it implies that none of its frequent super-graphs carries the same support

- *Lossless compression*: Does not contain non-closed graphs, but still ensures that the mining result is complete
- Algorithm CloseGraph: Mines closed graph patterns directly

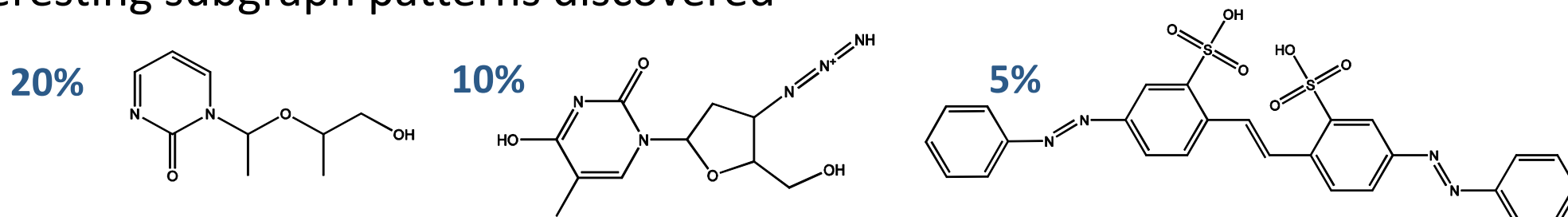
CloseGraph: Directly Mining Closed Graph Patterns

- CloseGraph: Mining closed graph patterns by extending gSpan (Yan & Han, KDD'03)

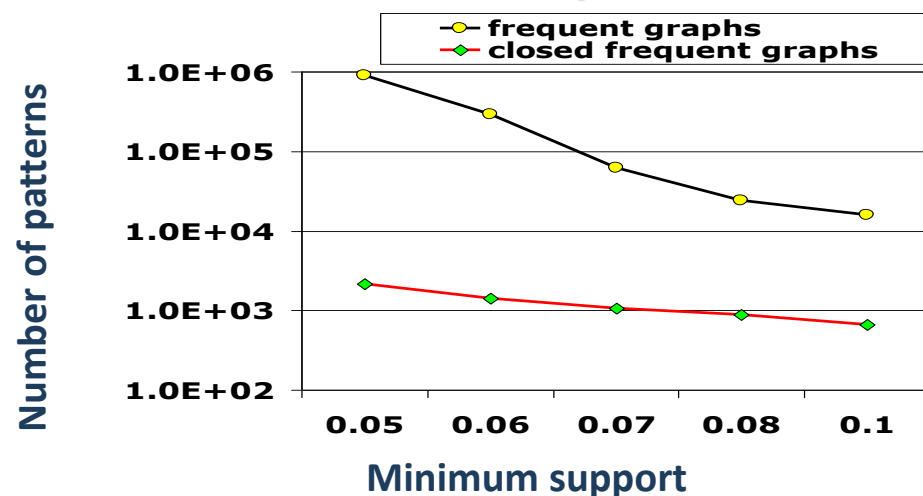


Experiment and Performance Comparison

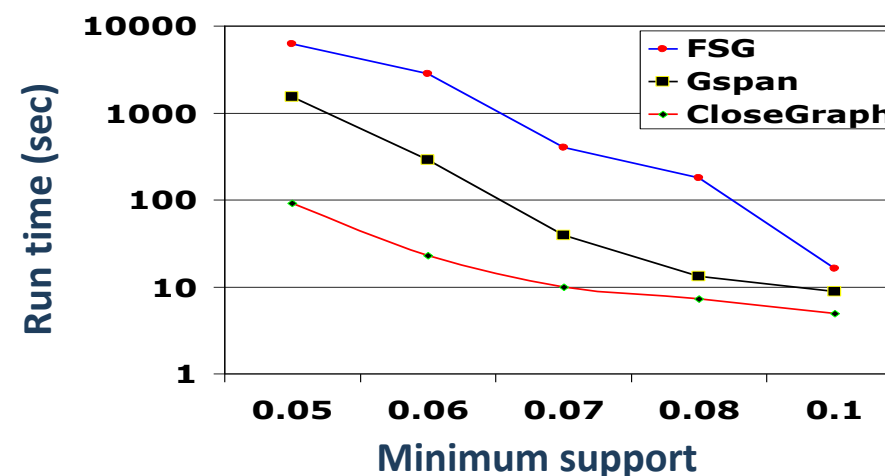
- ❑ The AIDS antiviral screen compound dataset from NCI/NIH
- ❑ The dataset contains 43,905 chemical compounds
- ❑ Discovered patterns: The smaller minimum support, the bigger and more interesting subgraph patterns discovered



of Patterns: Frequent vs. Closed

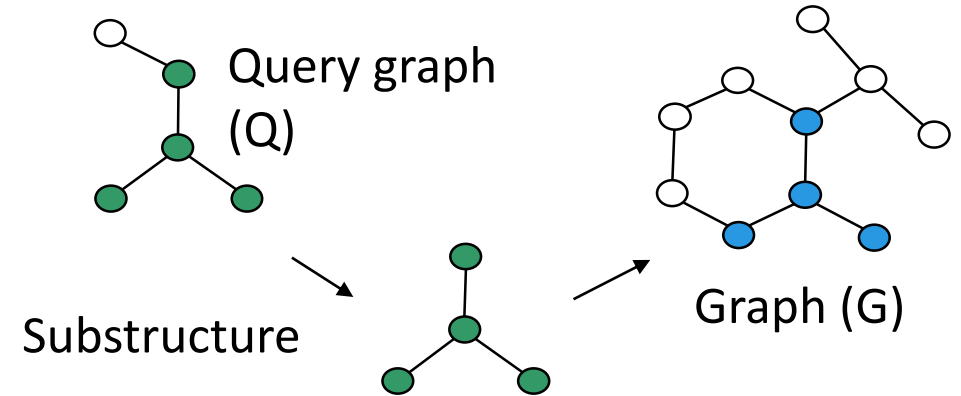


Runtime: Frequent vs. Closed

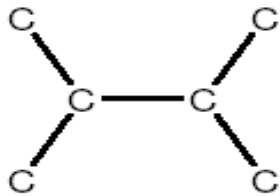


Application of Pattern Mining I: Graph Indexing

- ❑ Graph query: Find all the graphs in a graph DB containing a given query graph
- ❑ Index should be a powerful tool
- ❑ Path-index may not work well
- ❑ Solution: Index directly on substructures (i.e., graphs)

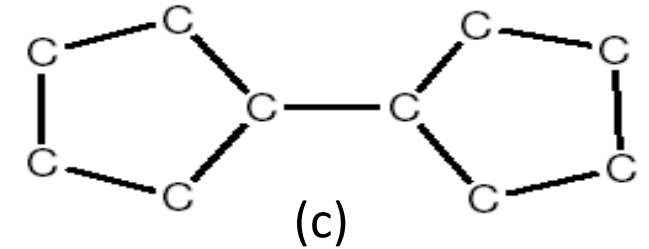
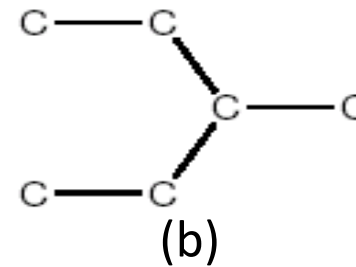
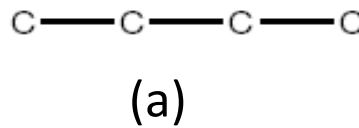


Query Q:



Only graph (c) contains Q

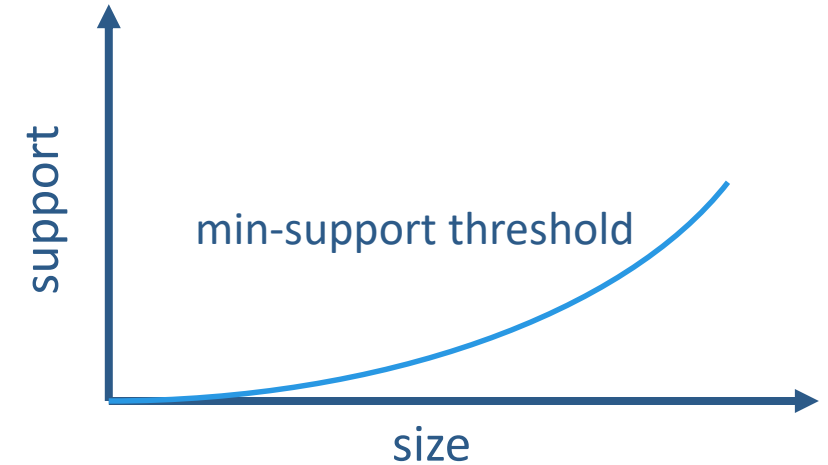
Graph DB:



Path-indices: C, C-C, C-C-C, C-C-C-C cannot prune (a) & (b)

gIndex: Indexing Frequent and Discriminative Substructures

- Why index frequent substructures?
 - Too many substructures to index
 - Size-increasing support threshold
 - Large structures will likely be indexed well by their substructures
- Why discriminative substructures?
 - Reduce the index size by an order of magnitude
- Selection: Given a set of selected structures $f_1, f_2, \dots, f_n$, and a new structure x , the extra indexing power is measured by
$$\Pr(x | f_1, f_2, \dots, f_n), f_i \subset x$$
when $\Pr(x | f_1, f_2, \dots, f_n)$ is small enough, x is a discriminative structure and should be included in the index
- Experiments show that gIndex is small, effective, and stable

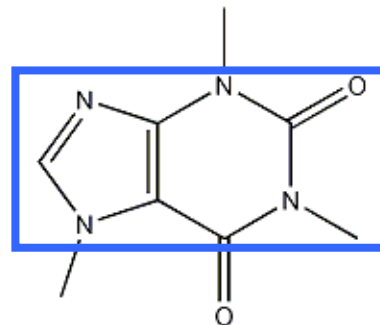
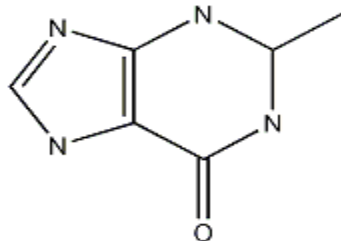


Application II: Support Substructure Similarity Search

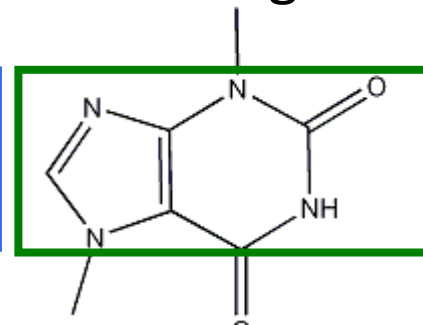
- Find graphs in a graph DB containing substructures similar to a given query graph

- Ex. Data: A chemical compound DB

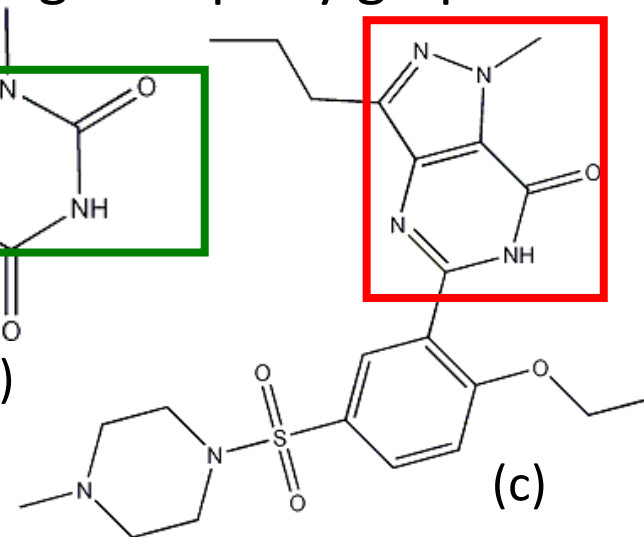
- A query graph q :



(a)



(b)



(c)

- How to do similarity search efficiently?

- No indexing? – Sequential scan + computing subgraph similarity – too costly!

- Build graph indices to support approximate search?

- Need an explosive number of subgraphs to cover all the *similar* subgraphs!

- An elegant solution (Yan, Yu, & Han, SIGMOD'05):

- Keep the graph index structure, but **select features** in the **query space**

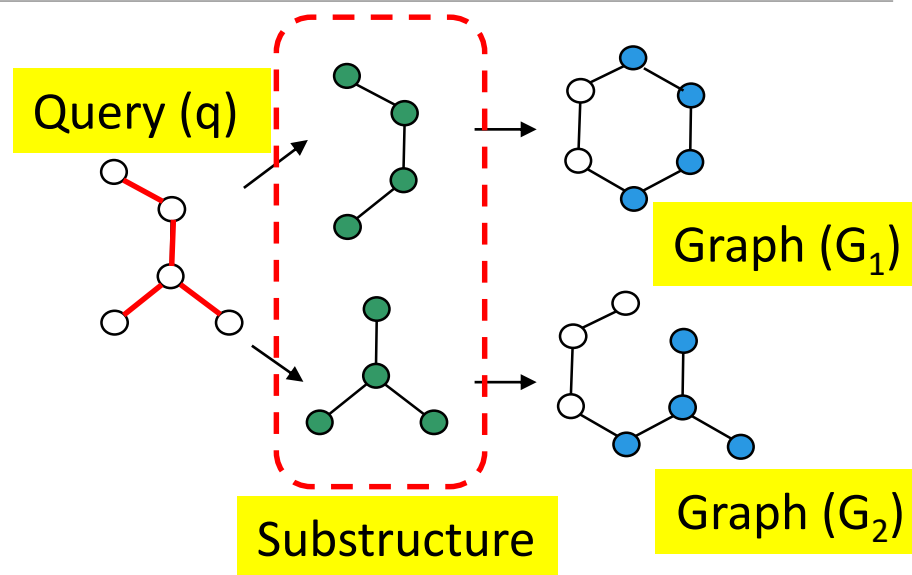
Feature-Based Similarity Search

- Decompose a query graph into a set of features
- Feature-based similarity measure
 - Each graph is represented as a feature vector
 $X = \{x_1, x_2, \dots, x_n\}$
 - Similarity is defined by the distance of their corresponding vectors
- If graph G contains the major part of a query graph q, G should share a number of common features with q
 - Given a relaxation ratio, one can calculate the maximal number of features that can be missed!

Assume: Query graph has 5 features

Relaxation threshold: Can miss at most 2 features

Then: G_1, G_2, G_3 are pruned



Graphs in database

	G_1	G_2	G_3	G_4	G_5
f_1	0	1	0	1	1
f_2	0	1	0	0	1
f_3	1	0	1	1	1
f_4	1	0	0	0	1
f_5	0	0	1	1	0

features

A feature-graph matrix

Pattern Mining: Advanced Methods

- ☐ Mining Various Kinds of Patterns
- ☐ Mining Compressed or Approximate Patterns
- ☐ Constraint-based Pattern Mining
- ☐ Mining Sequential Patterns
- ☐ Mining Subgraph Patterns
- ☐ Pattern Mining: Application Examples 
- ☐ Summary

Pattern Mining Application I: Software Bug Detection

❑ Mining rules from source code

- ❑ Bugs as deviant behavior (e.g., by statistical analysis)
- ❑ Mining programming rules (e.g., by frequent itemset mining)
- ❑ Mining function precedence protocols (e.g., by frequent subsequence mining)
- ❑ Revealing neglected conditions (e.g., by frequent itemset/subgraph mining)

❑ Mining rules from revision histories

- ❑ By frequent itemset mining

❑ Mining copy-paste patterns from source code

- ❑ Find copy-paste bugs (e.g., CP-Miner [Li et al., OSDI'04]) (to be discussed here)
 - ❑ **Reference:** Z. Li, S. Lu, S. Myagmar, Y. Zhou, “[CP-Miner](#): A Tool for Finding Copy-paste and Related Bugs in Operating System Code”, OSDI'04

Application Example: Mining Copy-and-Paste Bugs

- ❑ Copy-pasting is common
 - ❑ 12% in Linux file system
 - ❑ 19% in X Window system
- ❑ Copy-pasted code is error-prone
- ❑ Mine “*forget-to-change*” bugs by sequential pattern mining
 - ❑ Build a sequence database from source code
 - ❑ Mining sequential patterns
 - ❑ Finding mismatched identifier names & bugs

```
void __init prom_meminit(void)
{
    .....
    for (i=0; i<n; i++) {
        total[i].adr = list[i].adr;
        total[i].bytes = list[i].size;
        total[i].more = &total[i+1];
    }
    .....
```

```
for (i=0; i<n; i++) {
    taken[i].adr = list[i].adr;
    taken[i].bytes = list[i].size;
    taken[i].more = &total[i+1];
}
```

Code copy-and-pasted but **forget to change “id”!**

Courtesy of Yuanyuan Zhou@UCSD

(Simplified example from *linux-2.6.6/arch/sparc/prom/memory.c*)

Building Sequence Database from Source Code

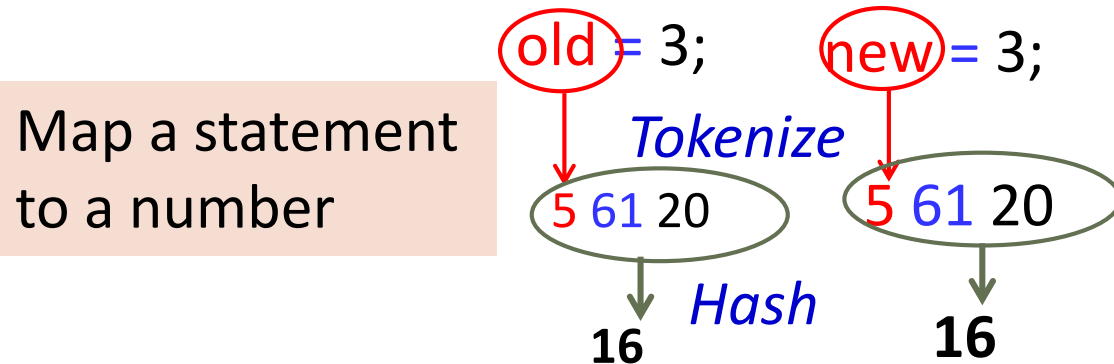
- Statement ^(mapped to) → number
- Tokenize each component
 - Different operators, constants, key words → different tokens
 - Same type of identifiers → same token
- Program → A long sequence
 - Cut the long sequence by blocks

Hash values

65	for (i=0; i<n; i++) {
16	total[i].adr = list[i].addr;
16	total[i].bytes = list[i].size;
71	total[i].more = &total[i+1];
	}
...	
65	for (i=0; i<n; i++) {
16	taken[i].adr = list[i].addr;
16	taken[i].bytes = list[i].size;
71	taken[i].more = &total[i+1];
	}

Final sequence DB:

(65)
(16, 16, 71)
...
(65)
(16, 16, 71)



Sequential Pattern Mining & Detecting “Forget-to-Change” Bugs

- Modification to the *sequence pattern mining algorithm*

- Constrain the max gap

(16, 16, 71)

.....

(16, 16, 10, 71)

Allow a maximal gap:
inserting statements
in copy-and-paste

- Composing Larger Copy-Pasted Segments

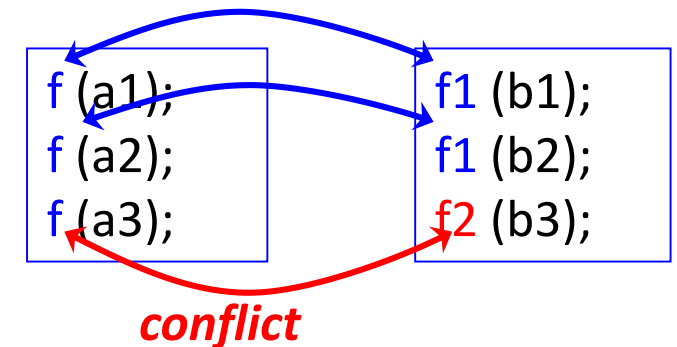
- Combine the neighboring copy-pasted segments repeatedly

- Find conflicts: Identify names that cannot be mapped to the corresponding ones

- E.g., 1 out of 4 “**total**” is unchanged, *unchanged ratio* = 0.25

- If $0 < \text{unchanged ratio} < \text{threshold}$, then report it as a bug

- CP-Miner reported many C-P bugs in Linux, Apache, ... out of millions of LOC (lines of code)



Why Phrase Mining?

- ❑ Unigrams vs. phrases
 - ❑ **Unigrams** (single words) are often *ambiguous*
 - ❑ Example: “United”: United States? United Airline? United Parcel Service?
 - ❑ **Phrase**: A natural, meaningful, *unambiguous* semantic unit
 - ❑ Example: “United States” vs. “United Airline”
- ❑ Mining semantically meaningful phrases
 - ❑ Transform text data from *word granularity* to *phrase granularity*
 - ❑ Enhance the power and efficiency at manipulating unstructured data

From Frequent Pattern Mining to Phrase Mining

- General principle
 - Exploit information redundancy and data-driven criteria to determine phrase boundaries and salience
- Methodology: Exploring three ideas
 - Frequent pattern mining and colocation analysis
 - Phrasal segmentation
 - Quality phrase assessment
- Recent developments of phrase mining methods
 - ToPMine: Mining quality phrase without training (A. El-Kishky, et al., 2015)
 - SegPhrase: Mining quality phrase with tiny training sets (J. Liu, et al., 2015)
 - AutoPhrase: Mining quality phrases with distant supervision (e.g., Wikipedia) (Shang, et al., 2018)

ToPMine: Phrase Mining before Topic Modeling

- ❑ **ToPMine** [El-Kishky et al. VLDB'15]: Phrase mining, then phrase-based topic modeling
- ❑ Phrase mining
 - ❑ **Frequent *contiguous pattern* mining**: Extract candidate phrases and their counts
 - ❑ Agglomerative merging of adjacent unigrams as guided by a **significance score**
 - ❑ Document segmentation to count phrase occurrence
 - ❑ Calculate rectified (i.e., true) phrase frequency
 - ❑ Phrase ranking (using the criteria proposed in KERT)
 - ❑ Popularity, concordance, informativeness, completeness
- ❑ Phrase-based topic modeling
 - ❑ The mined bag-of-phrases are passed as input to PhraseLDA, an extension of LDA, that constrains all words in a phrase to each sharing the same latent topic

Phrase	Raw frequency	Rectified frequency
[support vector machine]	90	80
[vector machine]	95	0
[support vector]	100	5

Collocation Mining

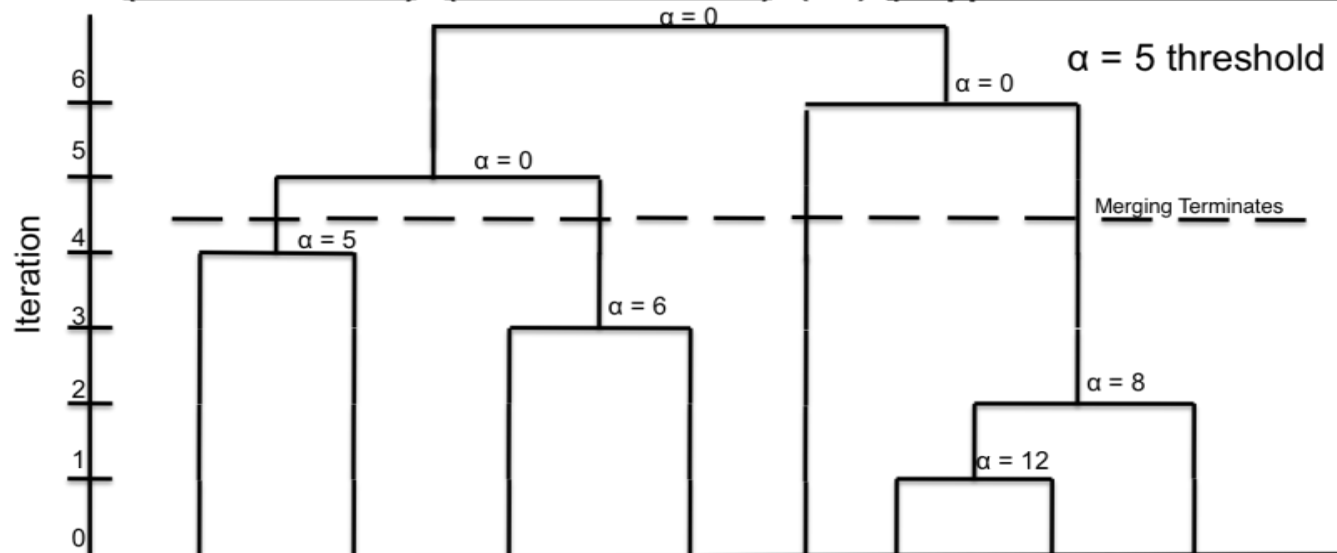
- Collocation: A sequence of words that occur **more frequently than expected**
 - Often “interesting”, relay information not portrayed by their constituent terms
 - Ex. “made an exception”, “strong tea”
- Many different measures used to extract collocations from a corpus [Dunning 93, Pederson 96]
 - E.g., mutual information, t-test, z-test, chi-squared test, likelihood ratio

$$\text{PMI}(x, y) = \log \frac{p(x, y)}{p(x)p(y)} \quad \text{sig} = \frac{\text{count}(\text{phr}_{x+y}) - E[\text{count}(\text{phr}_{x+y})]}{\sqrt{\text{count}(\text{phr}_{x+y})}} \quad \chi^2 = \sum \frac{(O - E)^2}{E}$$

- Many of these measures can be used to guide the agglomerative **phrase-segmentation** algorithm

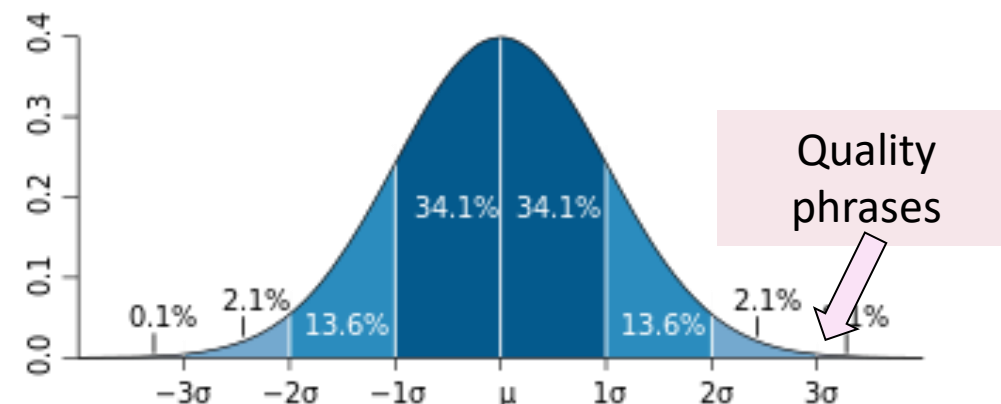
Phrase Candidate Generation: Frequent Pattern Mining + Statistical Analysis

(Markov Blanket) (Feature Selection) (for) (Support Vector Machines)



Markov Blanket Feature Selection for Support Vector Machines.

[Markov blanket] [feature selection] for [support vector machines]
[knowledge discovery] using [least squares] [support vector machine] [classifiers]
...[support vector] for [machine learning]...



Based on significance score [Church et al.'91]:

$$\alpha(P_1, P_2) \approx (f(P_1 \bullet P_2) - \mu_0(P_1, P_2)) / \sqrt{f(P_1 \bullet P_2)}$$

Note for the first title:

- ❑ [feature selection] forms phrase but not [selection for] based on the significant scores computed
- ❑ [support vector machine] does not contribute to the counts of [support], [vector], [support vector], [vector machine]

ToPMine: Experiments on DBLP Abstracts

	<i>Topic 1</i>	<i>Topic 2</i>	<i>Topic 3</i>	<i>Topic 4</i>	<i>Topic 5</i>
unigrams	problem algorithm optimal solution search solve constraints programming heuristic genetic	word language text speech system recognition character translation sentences grammar	data method algorithm learning clustering classification based features proposed classifier	programming language code type object implementation system compiler java data	data patterns mining rules set event time association stream large
n-grams	genetic algorithm optimization problem solve this problem optimal solution evolutionary algorithm local search search space optimization algorithm search algorithm objective function	natural language speech recognition language model natural language processing machine translation recognition system context free grammars sign language recognition rate character recognition	data sets support vector machine learning algorithm machine learning feature selection paper we propose clustering algorithm decision tree proposed method training data	programming language source code object oriented type system data structure program execution run time code generation object oriented programming java programs	data mining data sets data streams association rules data collection time series data analysis mining algorithms spatio temporal frequent itemsets

ToPMine is efficient and generates high-quality topics and phrases without any training data

ToPMine: Experiments on Yelp Reviews

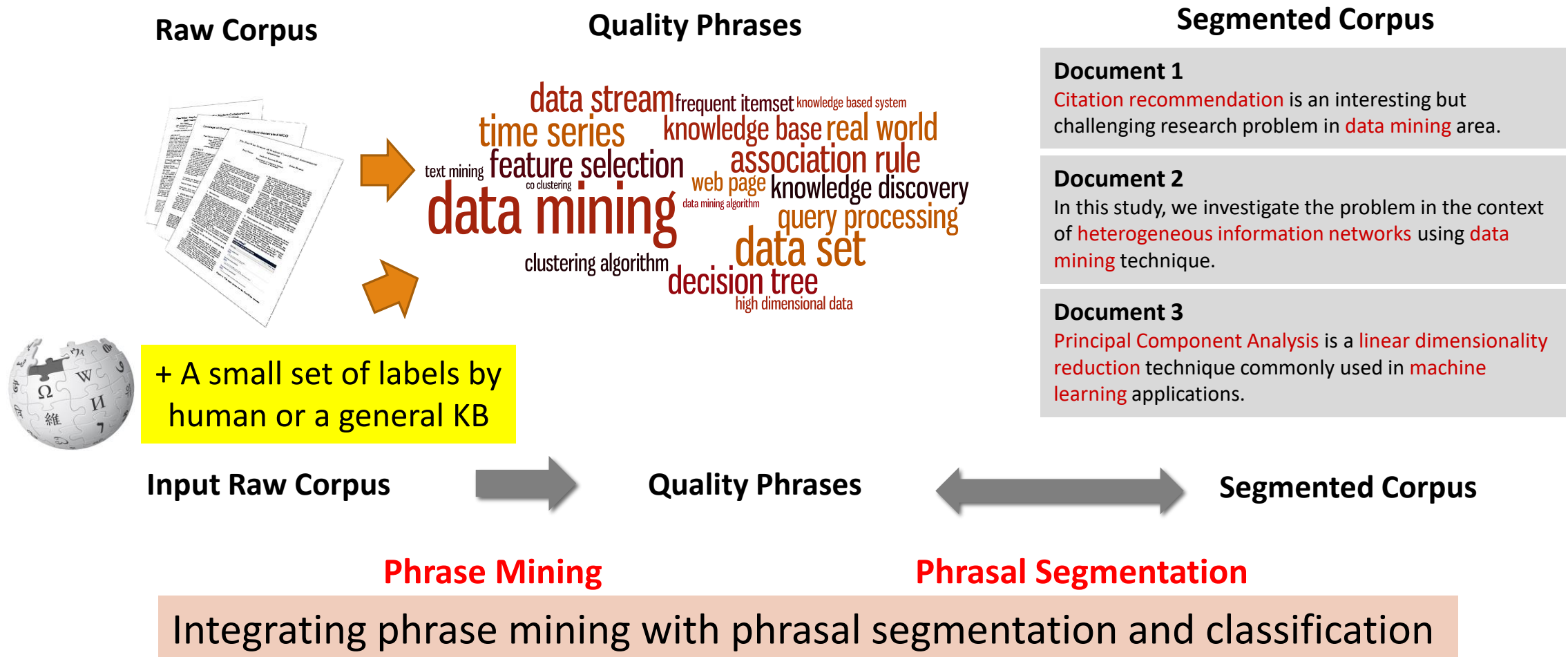
	<i>Topic 1</i>	<i>Topic 2</i>	<i>Topic 3</i>	<i>Topic 4</i>	<i>Topic 5</i>
unigrams	coffee ice cream flavor egg chocolate breakfast tea cake sweet	food good place ordered chicken roll sushi restaurant dish rice	room parking hotel stay time nice place great area pool	store shop prices find place buy selection items love great	good food place burger ordered fries chicken tacos cheese time
n-grams	ice cream iced tea french toast hash browns frozen yogurt eggs benedict peanut butter cup of coffee iced coffee scrambled eggs	spring rolls food was good fried rice egg rolls chinese food pad thai dim sum thai food pretty good lunch specials	parking lot front desk spring training staying at the hotel dog park room was clean pool area great place staff is friendly free wifi	grocery store great selection farmer's market great prices parking lot wal mart shopping center great place prices are reasonable love this place	mexican food chips and salsa food was good hot dog rice and beans sweet potato fries pretty good carne asada mac and cheese fish tacos

ToPMine works well for phrase and topic mining in social media data

SagPhrase: Phrase Mining with Tiny Training Sets

- A small set of training data may enhance the quality of phrase mining

J. Liu et al., Mining Quality Phrases from Massive Text Corpora. In *SIGMOD'15*

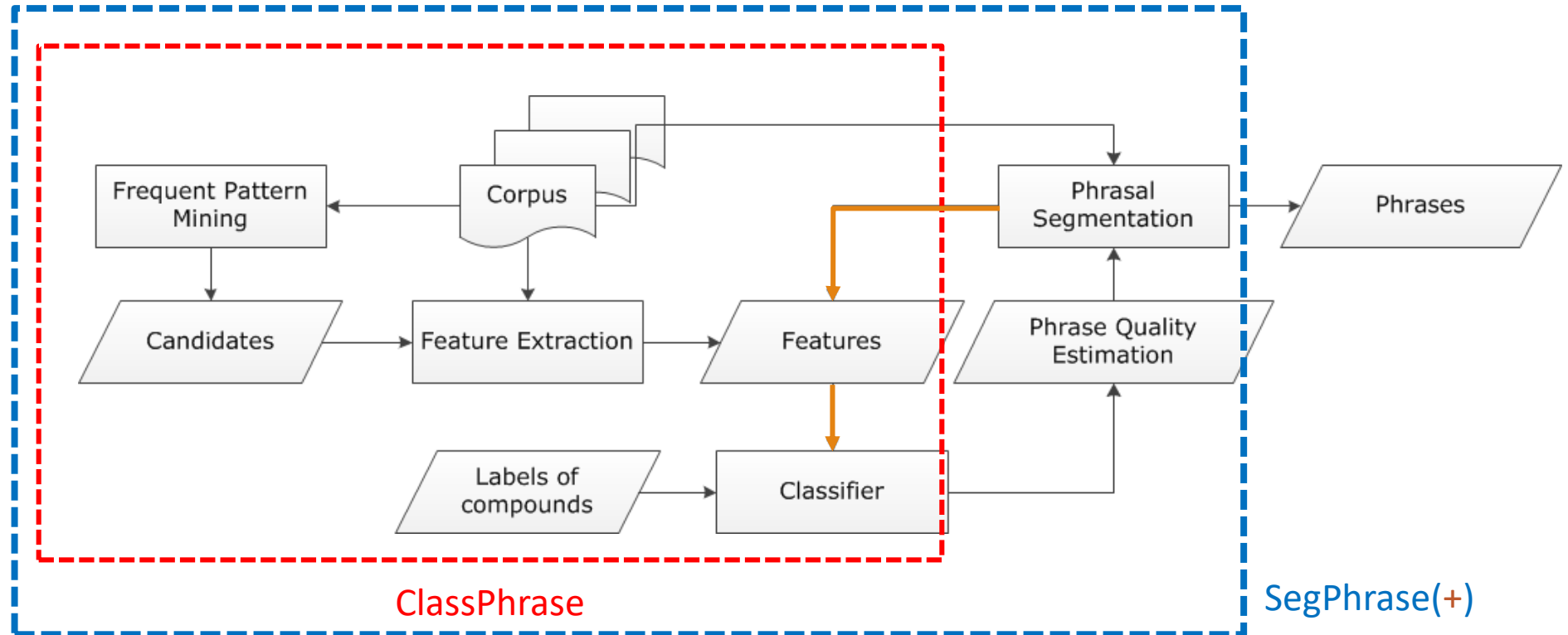


SegPhrase+: The Overall Framework

- ❑ ClassPhrase: Frequent pattern mining, feature extraction, classification
- ❑ SegPhrase: Phrasal segmentation and phrase quality estimation
- ❑ SegPhrase+: One more round to enhance mined phrase quality

SegPhrase (a classifier is used)

Small labeled dataset provided by experts or a distant supervised KB (e.g., Wikipedia / DBPedia)



SegPhrase: Pattern Mining and Feature Extraction

❑ Pattern Mining for Candidate Set

- ❑ Build a candidate phrases set by frequent pattern mining
 - ❑ Mining frequent k -grams (k is typically small, e.g., 6 in the experiments)
 - ❑ **Popularity** measured by *raw* frequent words and phrases mined from the corpus

❑ Feature Extraction: Concordance

- ❑ Partition a phrase into two parts to check whether the co-occurrence is significantly higher than pure random

❑ Feature Extraction: Informativeness

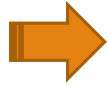
- ❑ Quality phrases typically start and end with a non-stopword
 - ❑ “machine learning is” vs. “machine learning”
- ❑ Use average IDF over words in the phrase to measure the semantics
- ❑ Usually, the probabilities of a quality phrase in quotes, brackets, or connected by hyphen should be higher (punctuations information)
 - ❑ e.g., “state-of-the-art”

SegPhrase: Classification Using Tiny Training Sets

- ❑ Use tiny training sets (300 labels for 1GB corpus; can also use phrases extracted from KBs)
 - ❑ Label: indicating whether a phrase is a high quality one
 - ❑ E.g., “support vector machine”: 1; “the experiment shows”: 0
- ❑ Classification: Construct models to distinguish quality phrases from poor ones
 - ❑ Use *Random Forest* algorithm to bootstrap different datasets with limited labels
- ❑ Phrasal segmentation can tell which phrase is more appropriate
 - ❑ Ex: “A standard [feature vector] [machine learning] setup is used to describe”
 - Not counted towards the rectified frequency
 - ❑ Partition a sequence of words by maximizing the likelihood
 - ❑ Consider length penalty and filter out phrases with low rectified frequency
- ❑ Process: Classification → Phrasal segmentation // **SegPhrase**
 - Classification → Phrasal segmentation // **SegPhrase+**

Performance: Precision Recall Curves on DBLP

□ Datasets:



□ Evaluation

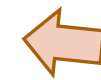
- Wiki Phrases (based on internal links, ~7K high quality phrases)
- Sampled 500*7 Wiki-uncovered phrases: Results evaluated by 3 reviewers

□ Compared with other phrase-mining methods

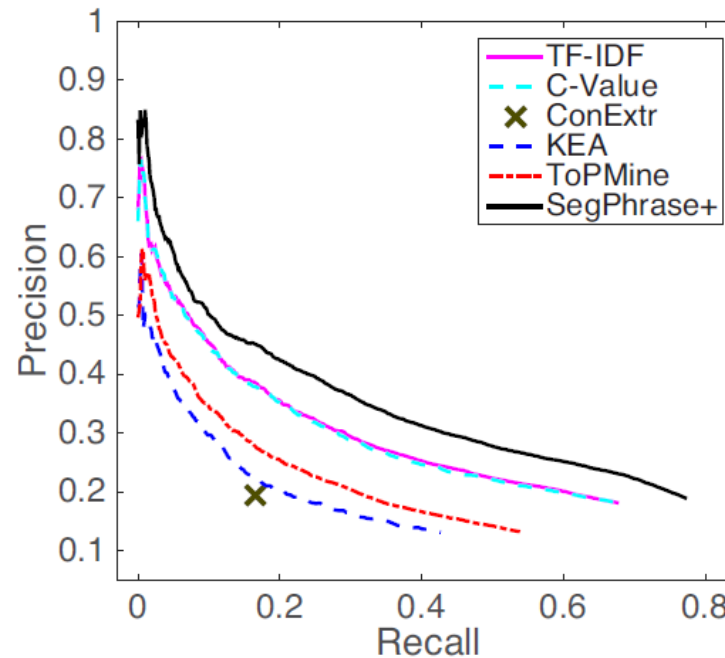
- TF-IDF, C-Value, ConExtr, KEA, and ToPMine

□ Also, Segphrase+ is efficient, linearly scalable

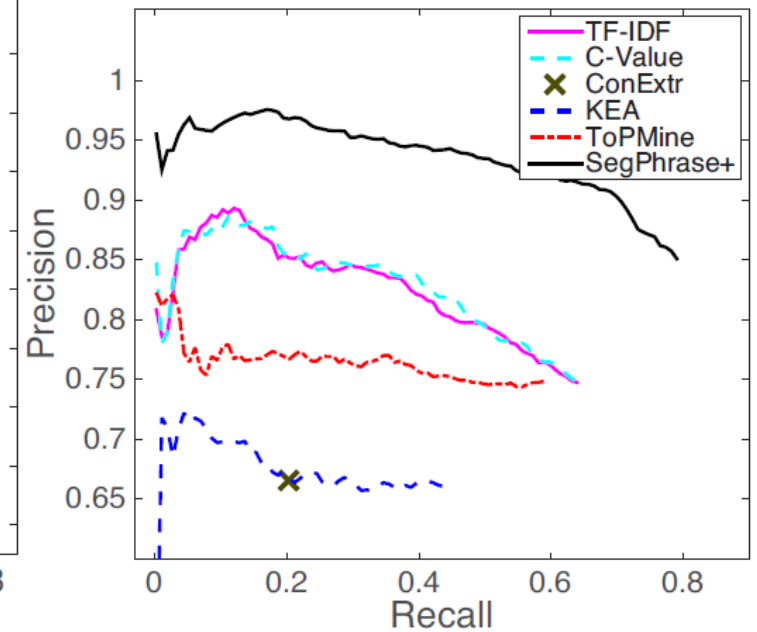
Dataset	#docs	#words	#labels
DBLP	2.77M	91.6M	300
Yelp	4.75M	145.1M	300



Use only 300 human labeled phrases for training



Precision-Recall Curves on DBLP Data (Wiki Phrases)



Precision-Recall Curves on DBLP Data (Non Wiki-phrases)

Experimental Results: Interesting Phrases Generated (From Titles & Abstracts of SIGKDD)

Query	SIGKDD	
Method	SegPhrase+	Chunking (TF-IDF & C-Value)
1	data mining	data mining
2	data set	association rule
3	association rule	knowledge discovery
4	knowledge discovery	frequent itemset
5	time series	decision tree
...	...	...
51	association rule mining	search space
52	rule set	domain knowledge
53	concept drift	important problem
54	knowledge acquisition	concurrency control
55	gene expression data	conceptual graph
...	...	...
201	web content	optimal solution
202	frequent subgraph	semantic relationship
203	intrusion detection	effective way
204	categorical attribute	space complexity
205	user preference	small set
...	...	...

Only in Chunking

Only in SegPhrase+

Mining Quality Phrases in Multiple Languages

- Both ToPMine and SegPhrase+ are extensible to mining quality phrases in multiple languages

- SegPhrase+ on Chinese (From Chinese Wikipedia)



- ToPMine on Arabic (From Quran (Fus7a Arabic)(no preprocessing)

- Experimental results of Arabic phrases:

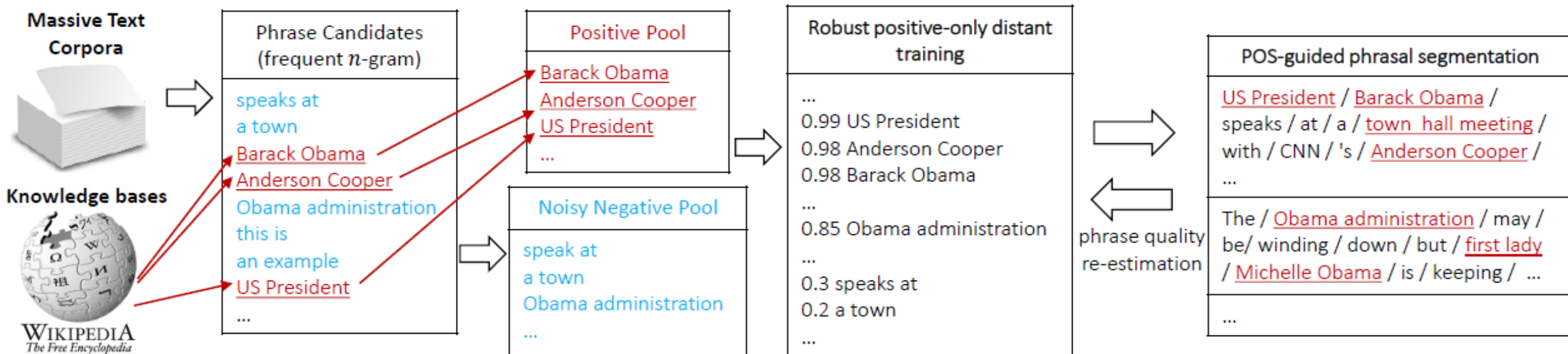
كفروا → Those who disbelieve

بسم الله الرحمن الرحيم → In the name of God the Gracious and Merciful

Rank	Phrase	In English
...	...	...
62	首席_执行官	CEO
63	中间_偏右	Middle-right
...	...	...
84	百度_百科	Baidu Pedia
85	热带_气旋	Tropical cyclone
86	中国科学院_院士	Fellow of Chinese Academy of Sciences
...	...	...
1001	十大_中文_金曲	Top-10 Chinese Songs
1002	全球_资讯网	Global News Website
1003	天一阁_藏_明代_科举_录_选刊	A Chinese book name
...	...	...
9934	国家_戏剧_院	National Theater
9935	谢谢_你	Thank you
...	...	...

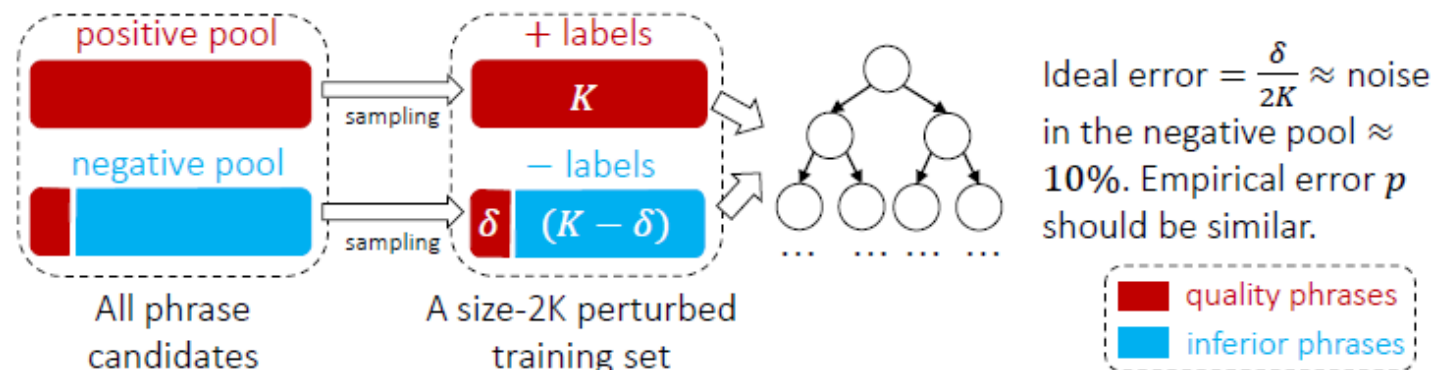
AutoPhrase: Automated Phrase Mining by Distant Supervision

- ❑ AutoPhrase: *Automatic* extraction of high-quality phrases (e.g., scientific terms and general entity names) in a given corpus (e.g., research papers and news)
- ❑ Major features:
 - ❑ No human efforts; multiple languages; high performance—precision, recall, efficiency
 - ❑ **Distant training:** Utilize quality phrases in KBs (e.g., Wiki) as *positive* phrase labels
- ❑ **Innovation:** Sampling-based label generation for robust, positive-only distant training



Robust Positive-Only Distant Training

- In each base classifier, randomly sample K *positive* (e.g., wiki titles, keywords, links) and K *noisy negative labels* from the pools



- Noisy negative pool: may still have δ quality phrases among the K negative labels
- They form “perturbed training set”: size- $2K$ subset of the full set of all phrases where the labels of some quality phrases are switched from positive to negative
- Each base classifier can be viewed as randomly drawn K phrase candidates with replacement from the positive pool and the negative pool respectively
 - Grow an unpruned decision tree to the point of separating all phrases to meet this requirement
- Use an *ensemble classifier* that averages the results of independently trained base classifiers

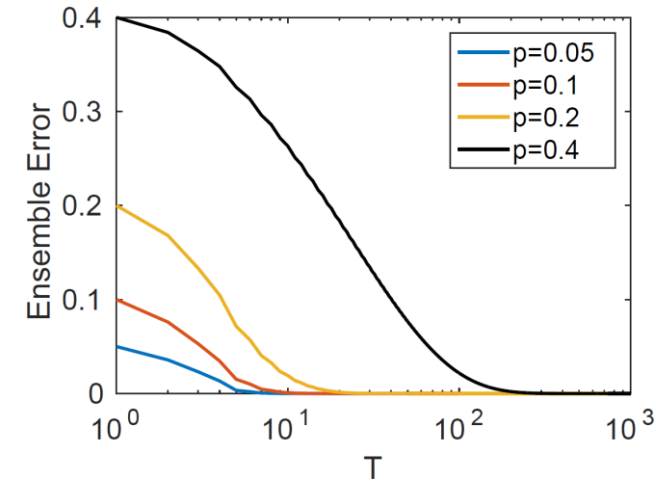
Why Is Positive-Only Distant Training Robust?

□ Theoretical Analysis

□ T base classifiers

$$\text{ensemble_error}(T) = \sum_{t=\lfloor 1+T/2 \rfloor}^T \binom{T}{t} p^t (1-p)^{T-t}$$

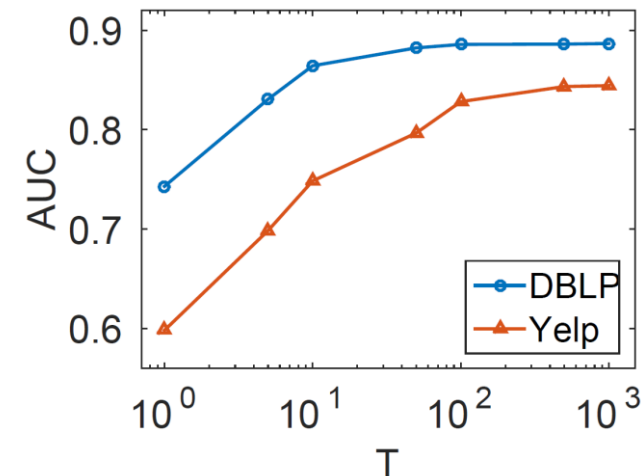
□ Exponentially decreasing



□ Empirical Performance

□ AUC to evaluate the ranking

Note: AUC (Area Under Curve), with value range [0,1], is a classification measure to be introduced in the classification module

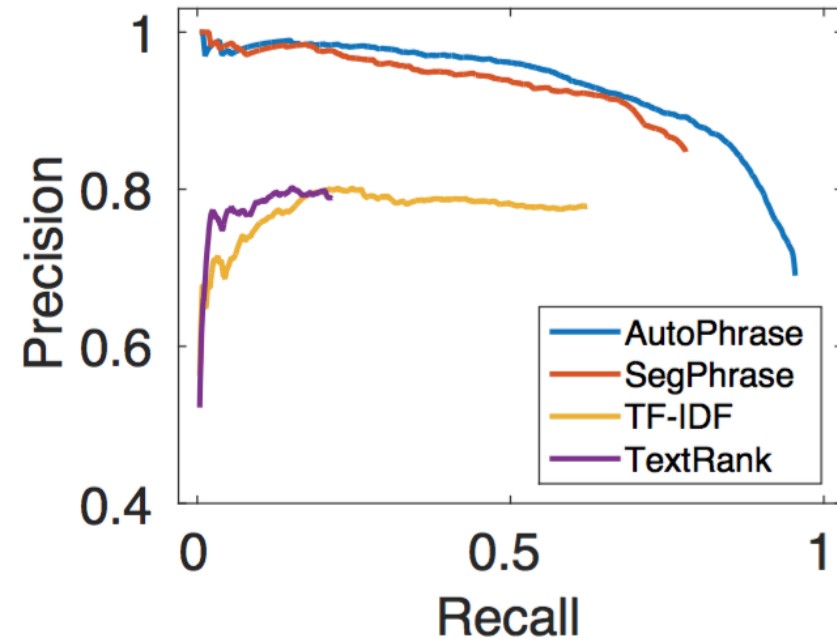


Modeling Single-Word Phrases: Enhancing Recall

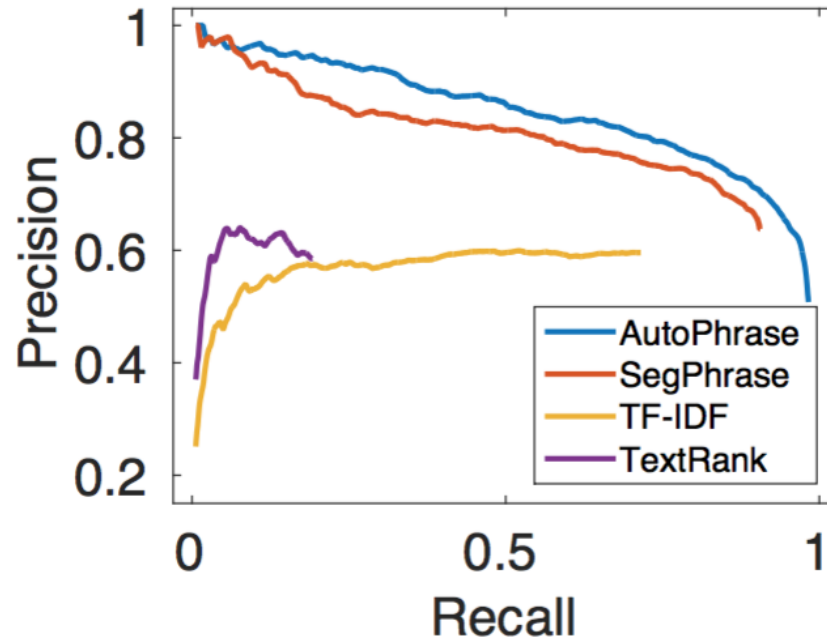
- ❑ AutoPhrase simultaneously models single-word and multi-word phrases
 - ❑ A phrase can also be a single word, as long as it functions as a constituent in the syntax of a sentence, e.g., “UIUC”, “Illinois”
 - ❑ Based on our experiments: 10%~30% quality phrases are single-word phrases
- ❑ Criteria for modeling single-word phrases
 - ❑ **Popularity**: Sufficiently frequent in a given corpus
 - ❑ **Informativeness**: Indicative of a specific topic or concept
 - ❑ **Independence**: A quality single-word phrase is more likely a complete semantic unit in a given document
- ❑ Example: Is the following good single-word phrase?
 - ❑ “CMU”? Yes (frequent, informative, independent)
 - ❑ “this”? No (not informative)
 - ❑ “united”? No (not independent, may be in “United States”, “United Airline”,...)

AutoPhrase: Cross-Domain Evaluation Results

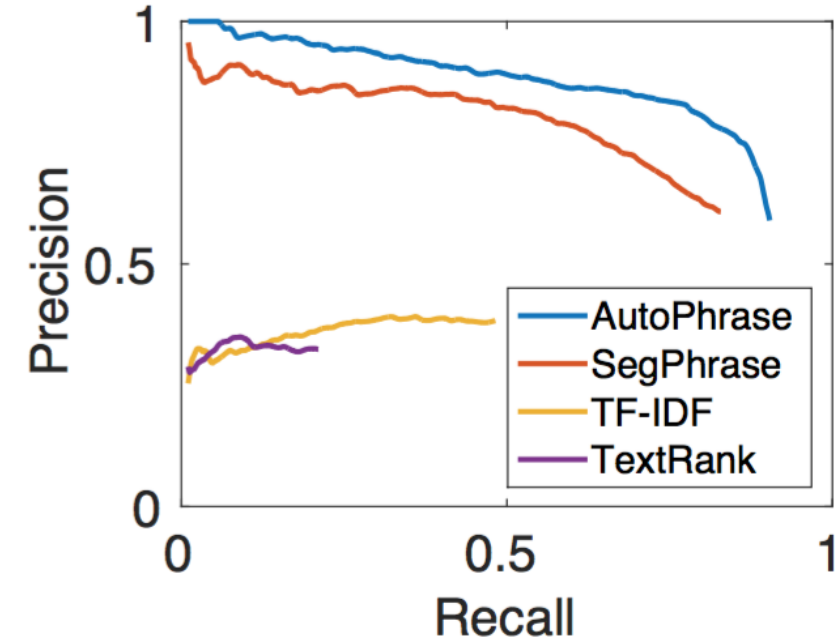
Computer Science Papers



Yelp Business Reviews



Wikipedia Articles



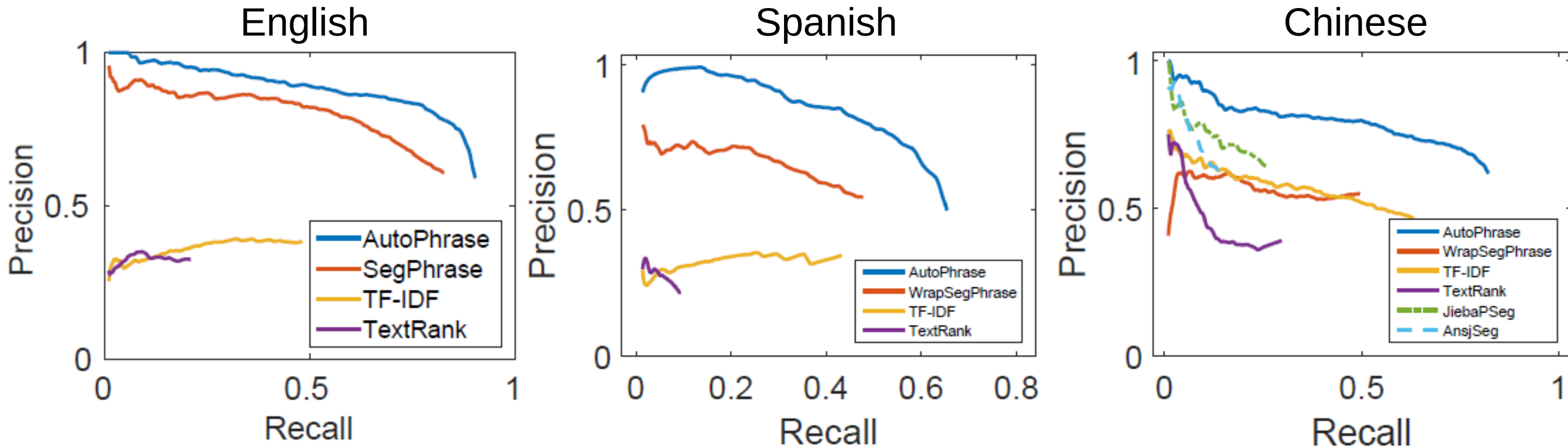
AutoPhrase (TKDE'18): Best performing and generating multi-word and single word phrases

SegPhrase (SIGMOD'15): Outperformed TopMine (VLDB'15) and many other methods

TF-IDF: Stanford NLP Parser (LREC'16) + Ranked by TF-IDF

TextRank (ACL'04): Stanford NLP Parser (LREC'16) + Ranked by TextRank

AutoPhrase: Cross-Language Evaluation Results



AutoPhrase (TKDE'18): Best performing and generating multi-word and single word phrases

WrapSegPhrase: non-English characters → English letters & SegPhrase

JiebaSeg: Specifically for Chinese; Dictionaries & Hidden Markov Models

AnsjSeg: Specifically for Chinese; Dictionaries & Conditional Random Fields

AutoPhrase: An Example Run From Chinese Wikipedia

Phrase's Rank	Phrase	Translation (Explanation)
1	江苏_舜_天	(the name of a soccer team)
2	苦_艾_酒	Absinthe
3	白发_魔_女	(the name of a novel/TV-series)
4	笔记_型_电脑	notebook computer, laptop

- ❑ The size of positive pool is about 29,000
- ❑ AutoPhrase finds more than 116,000 quality phrases (quality score > 0.5)

99,994	计算机_科学技术	Computer Science and Technology
99,995	恒_天然	Fonterra (a company)
99,996	中国_作家_协会_副_主席	The Vice President of Writers Association of China
99,997	维他命_b	Vitamin B
99,998	舆论_导向	controlled guidance of the media
...	...	...

Pattern Mining: Advanced Methods

- ☐ Mining Various Kinds of Patterns
- ☐ Mining Compressed or Approximate Patterns
- ☐ Constraint-based Pattern Mining
- ☐ Mining Sequential Patterns
- ☐ Mining Subgraph Patterns
- ☐ Pattern Mining: Application Examples
- ☐ Summary



Summary: Pattern Mining: Advanced Methods (I)

❑ Mining Diverse Patterns

- ❑ Mining Multiple-Level Associations
- ❑ Mining Multi-Dimensional Associations
- ❑ Mining Quantitative Associations
- ❑ Mining Negative Correlations
- ❑ Mining Compressed and Redundancy-Aware Patterns

❑ Constraint-Based Frequent Pattern Mining

- ❑ Why Constraint-Based Mining?
- ❑ Constrained Mining with Pattern Anti-Monotonicity
- ❑ Constrained Mining with Pattern Monotonicity
- ❑ Constrained Mining with Data Anti-Monotonicity
- ❑ Constrained Mining with Succinct Constraints
- ❑ Constrained Mining with Convertible Constraints
- ❑ Handling Multiple Constraints
- ❑ Constraint-Based Sequential-Pattern Mining

Summary: Pattern Mining: Advanced Methods (II)

☐ Sequential Pattern Mining

- ☐ Sequential Pattern and Sequential Pattern Mining
- ☐ GSP: Apriori-Based Sequential Pattern Mining
- ☐ SPADE: Sequential Pattern Mining in Vertical Data Format
- ☐ PrefixSpan: Sequential Pattern Mining by Pattern-Growth
- ☐ CloSpan: Mining Closed Sequential Patterns
- ☐ Constraint-Based Sequential Pattern Mining

☐ Graph Pattern Mining

- ☐ Graph Pattern and Graph Pattern Mining
- ☐ Apriori-Based Graph Pattern Mining Methods
- ☐ gSpan: A Pattern-Growth-Based Method
- ☐ CloseGraph: Mining Closed Graph Patterns
- ☐ Graph Pattern Applications
- ☐ Application I: Mining Software Copy-and-Paste Bugs
- ☐ Application II: Phrase Mining

References: Mining Diverse Patterns

- ❑ R. Srikant and R. Agrawal, “Mining generalized association rules”, VLDB'95
- ❑ Y. Aumann and Y. Lindell, “A Statistical Theory for Quantitative Association Rules”, KDD'99
- ❑ K. Wang, Y. He, J. Han, “Pushing Support Constraints Into Association Rules Mining”, IEEE Trans. Knowledge and Data Eng. 15(3): 642-658, 2003
- ❑ D. Xin, J. Han, X. Yan and H. Cheng, "On Compressing Frequent Patterns", Knowledge and Data Engineering, 60(1): 5-29, 2007
- ❑ D. Xin, H. Cheng, X. Yan, and J. Han, "Extracting Redundancy-Aware Top-K Patterns", KDD'06
- ❑ J. Han, H. Cheng, D. Xin, and X. Yan, "Frequent Pattern Mining: Current Status and Future Directions", Data Mining and Knowledge Discovery, 15(1): 55-86, 2007
- ❑ F. Zhu, X. Yan, J. Han, P. S. Yu, and H. Cheng, “Mining Colossal Frequent Patterns by Core Pattern Fusion”, ICDE'07

References: Constraint-Based Frequent Pattern Mining

- ❑ R. Srikant, Q. Vu, and R. Agrawal, “Mining association rules with item constraints”, KDD'97
- ❑ R. Ng, L.V.S. Lakshmanan, J. Han & A. Pang, “Exploratory mining and pruning optimizations of constrained association rules”, SIGMOD'98
- ❑ G. Grahne, L. Lakshmanan, and X. Wang, “Efficient mining of constrained correlated sets”, ICDE'00
- ❑ J. Pei, J. Han, and L. V. S. Lakshmanan, “Mining Frequent Itemsets with Convertible Constraints”, ICDE'01
- ❑ J. Pei, J. Han, and W. Wang, “Mining Sequential Patterns with Constraints in Large Databases”, CIKM'02
- ❑ F. Bonchi, F. Giannotti, A. Mazzanti, and D. Pedreschi, “ExAnte: Anticipated Data Reduction in Constrained Pattern Mining”, PKDD'03
- ❑ F. Zhu, X. Yan, J. Han, and P. S. Yu, “gPrune: A Constraint Pushing Framework for Graph Pattern Mining”, PAKDD'07

References: Sequential Pattern Mining

- ❑ R. Srikant and R. Agrawal, “Mining sequential patterns: Generalizations and performance improvements”, EDBT’96
- ❑ M. Zaki, “SPADE: An Efficient Algorithm for Mining Frequent Sequences”, Machine Learning, 2001
- ❑ J. Pei, J. Han, B. Mortazavi-Asl, J. Wang, H. Pinto, Q. Chen, U. Dayal, and M.-C. Hsu, "Mining Sequential Patterns by Pattern-Growth: The PrefixSpan Approach", IEEE TKDE, 16(10), 2004
- ❑ X. Yan, J. Han, and R. Afshar, “CloSpan: Mining Closed Sequential Patterns in Large Datasets”, SDM'03
- ❑ J. Pei, J. Han, and W. Wang, "Constraint-based sequential pattern mining: the pattern-growth methods", J. Int. Inf. Sys., 28(2), 2007
- ❑ M. N. Garofalakis, R. Rastogi, K. Shim: Mining Sequential Patterns with Regular Expression Constraints. IEEE Trans. Knowl. Data Eng. 14(3), 2002
- ❑ H. Mannila, H. Toivonen, and A. I. Verkamo, “Discovery of frequent episodes in event sequences”, Data Mining and Knowledge Discovery, 1997

References: Graph Pattern Mining

- ❑ C. Borgelt and M. R. Berthold, Mining molecular fragments: Finding relevant substructures of molecules, ICDM'02
- ❑ J. Huan, W. Wang, and J. Prins. Efficient mining of frequent subgraph in the presence of isomorphism, ICDM'03
- ❑ A. Inokuchi, T. Washio, and H. Motoda. An apriori-based algorithm for mining frequent substructures from graph data, PKDD'00
- ❑ M. Kuramochi and G. Karypis. Frequent subgraph discovery, ICDM'01
- ❑ S. Nijssen and J. Kok. A Quickstart in Frequent Structure Mining can Make a Difference. KDD'04
- ❑ N. Vanetik, E. Gudes, and S. E. Shimony. Computing frequent graph patterns from semistructured data, ICDM'02
- ❑ X. Yan and J. Han, gSpan: Graph-Based Substructure Pattern Mining, ICDM'02
- ❑ X. Yan and J. Han, CloseGraph: Mining Closed Frequent Graph Patterns, KDD'03
- ❑ X. Yan, P. S. Yu, J. Han, Graph Indexing: A Frequent Structure-based Approach, SIGMOD'04
- ❑ X. Yan, P. S. Yu, and J. Han, Substructure Similarity Search in Graph Databases, SIGMOD'05

References: Phrase Mining

- ❑ S. Bergsma, E. Pitler, D. Lin, [Creating Robust Supervised Classifiers via Web-scale N-gram Data](#), ACL'2010
- ❑ D. M. Blei and J. D. Lafferty. [Visualizing Topics with Multi-word Expressions](#). arXiv:0907.1013, 2009
- ❑ D.M. Blei, A. Y. Ng, M. I. Jordan, J. D. Lafferty, [Latent Dirichlet Allocation](#). JMLR 2003
- ❑ M. Danilevsky, C. Wang, N. Desai, X. Ren, J. Guo, J. Han. [Automatic Construction and Ranking of Topical Keyphrases on Collections of Short Documents](#). SDM'14
- ❑ A. El-Kishky, Y. Song, C. Wang, C. R. Voss, and J. Han. [Scalable Topical Phrase Mining from Text Corpora](#). VLDB'15
- ❑ R. V. Lindsey, W. P. Headden, III, M. J. Stipicevic. [A Phrase-Discovering Topic Model Using Hierarchical Pitman-Yor Processes](#). EMNLP-CoNLL'12.
- ❑ J. Liu, J. Shang, C. Wang, X. Ren, J. Han, [Mining Quality Phrases from Massive Text Corpora](#). SIGMOD'15
- ❑ A. Parameswaran, H. Garcia-Molina, and A. Rajaraman. [Towards the Web of Concepts: Extracting Concepts from Large Datasets](#). VLDB'10
- ❑ X. Wang, A. McCallum, X. Wei. [Topical N-grams: Phrase and Topic Discovery, With and Application to Information Retrieval](#). ICDM'07
- ❑ J. Shang, J. Liu, M. Jiang, X. Ren, C. R Voss, J. Han, "[Automated Phrase Mining from Massive Text Corpora](#)", IEEE Transactions on Knowledge and Data Engineering, 30(10):[1825-1837](#) (2018)